SQL Handbook — Interview Edition

How to use this edition

The book you re-read the night before an interview: every chapter of the *SQL Handbook* (PostgreSQL first, with MySQL, SQL Server, SQLite and Oracle differences) cut down to the facts, numbers, questions and traps that SQL and database interviews actually test. Nothing here is new — each line comes from the full book; open the matching chapter there (`../chapters/NN-name/README.md`) when you need the explanation, the runnable example or its real output.

- **One section per chapter (1–20)**, always in the same order: **Must-know** (one-line definitions) → **Key facts & numbers** (tables) → **Code to remember** (snippets you may be asked to write) → **In the interview** (likely questions, short answers) → **Gotchas** (mistake → fix) → **TL;DR**.
- **Night before:** read the *Top 25 questions* below, then every chapter's **In the interview** and **Gotchas**, then the *Last-minute checklist* at the end.
- **A week before:** read each chapter's **Must-know** and **Key facts**; say each **In the interview** answer out loud in 30 seconds; for anything shaky, go back to the full chapter.
- **Coding rounds:** Chapters 18–19 here give the method, the edge-case checklist, the full pattern table and the 19 most representative problems (problem → key idea → query). The other 64 worked problems are in the full book.
- **Design rounds:** Chapters 10, 11, 13 and 17 (modeling, normalization, transactions, case-study patterns), then 15 for scaling.
- All SQL is PostgreSQL unless a row says otherwise. Versions are marked where they matter: *PG 15+* means PostgreSQL 15 or later.

Top 25 questions across the book

#	Question	One-line answer	Ch
1	In what order is a SELECT evaluated?	FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT (windows) → DISTINCT → set ops → ORDER BY → LIMIT; so WHERE can't see aliases or windows	4
2	Why does `x = NULL` fail, and why can `NOT IN` return nothing?	NULL comparisons are UNKNOWN and WHERE keeps only TRUE; a NULL in the NOT IN list makes it never TRUE → `IS NULL`, `NOT EXISTS`	2, 6
3	WHERE vs HAVING?	Rows before grouping (no aggregates) vs groups after aggregation	4
4	`count(*)` vs `count(col)` vs `count(DISTINCT col)`?	Rows / non-NULL values / distinct non-NULL values; empty input → 0, other aggregates → NULL	4
5	Explain the join types.	Inner = matches; left/right = keep one side NULL-padded; full = both; cross = all pairs; self = two aliases on one table	5
6	Why did my LEFT JOIN act like an inner join?	A right-table condition in WHERE drops padded rows; move it into ON	5
7	Semi-join vs anti-join; IN vs EXISTS?	Has a match (EXISTS/IN, once each) vs has none (NOT EXISTS); IN/EXISTS plan the same, only NOT IN breaks on NULLs	5, 6
8	UNION vs UNION ALL?	UNION de-duplicates (sort/hash cost); UNION ALL appends — default to it	6
9	CTE vs subquery vs view vs materialized view?	Named step in one statement (inlined PG 12+) / inline / stored query always current / stored result refreshed by hand	7
10	How does a recursive CTE work?	Anchor rows, then the recursive term runs on the previous round's rows until a round is empty; cast in the anchor, guard cycles	7
11	Window function vs GROUP BY; ROW_NUMBER vs RANK vs DENSE_RANK?	Windows keep every row; 1,2,3,4 vs 1,2,2,4 vs 1,2,2,3 on a tie	8
12	Nth highest salary / top-N per group?	Scalar `SELECT DISTINCT … OFFSET n-1 LIMIT 1` (NULL if none) or `dense_rank() = n`; rank in a CTE, filter outside	8, 18, 19
13	Gaps and islands?	`value − row_number()` (or difference of two row numbers) is constant within a run → GROUP BY it	8, 19
14	DELETE vs TRUNCATE vs DROP?	Chosen rows, triggers fire / all rows fast, no WHERE / the table itself; all rollback-able in PG, TRUNCATE not in MySQL/Oracle	9
15	Primary vs unique vs foreign key; natural vs surrogate?	PK unique + not null, one per table; UNIQUE = alternate key, NULLs allowed; FK must match a key; surrogate PK + UNIQUE natural key	9, 10
16	1NF, 2NF, 3NF, BCNF; when denormalize?	Atomic / no partial dependency / no transitive dependency / every determinant a superkey; denormalize only for a measured read bottleneck with a maintenance path	11
17	How do you model many-to-many?	Junction table with both FKs as the PK; relationship attributes live there	10
18	How does a B+ tree index help; why isn't my index used?	Shallow sorted tree (3–4 levels), linked leaves for ranges; not used with functions/casts on the column, leading `%`, low selectivity, stale stats, non-leading composite column, tiny table	12

#	Question	One-line answer	Ch
19	Composite index column order; covering index?	Leftmost prefix, equality columns first, range last; `INCLUDE` columns for index-only scans	12
20	Why is OFFSET paging slow?	It reads and discards skipped rows; keyset `WHERE (ts, id) < (:ts, :id)` on a matching index	2, 12
21	Explain ACID and the isolation anomalies.	All-or-nothing, rules hold, no interference, survives crashes; dirty read, non-repeatable read, phantom, lost update, write skew	13
22	PostgreSQL isolation levels?	Read Committed default; Repeatable Read = snapshot isolation (no phantoms, aborts lost updates, allows write skew); Serializable = SSI, retry 40001	13
23	Prevent lost updates / build a job queue?	Atomic `SET x = x - 1`, `FOR UPDATE`, RR + retry, or version column; queue = `FOR UPDATE SKIP LOCKED LIMIT 1`	13
24	What is MVCC?	Row versions per transaction; readers see their snapshot and never block writers; VACUUM removes dead versions	13
25	Partitioning vs sharding vs replication; CAP?	Split a table on one server / partitions on many servers / copies of the same data; under a partition choose C or A (PACELC: else latency vs consistency)	15

Also high-frequency: SQL injection → parameterized queries (14); N+1 → one join or `= ANY($1)` (12, 14); join algorithms → nested loop / hash / merge (5, 12); backup vs replica → replicas copy mistakes, PITR restores (9, 15).

1. Databases and the Relational Model

Why databases replaced files, what a DBMS is made of, the relational model, relational vs NoSQL, and SQL's five sub-languages.

Must-know

- **File-era problems** — redundancy → inconsistency, data isolation, no ad-hoc queries, rules buried in code, lost updates, no crash recovery, all-or-nothing security. A DBMS fixes them by mediating every access.
- **Database** — related, self-describing (it stores **metadata** in a **catalog** / data dictionary), purposeful, persistent data. **DBMS** = the software; **database system** = DBMS + databases + applications.
- **DBMS jobs** — define, construct, manipulate, share, protect, maintain.
- **Physical data independence** — change storage (indexes, disks) without changing tables or queries. **Logical** — add a column/table without breaking apps that don't use it.
- **Relation / tuple / attribute / domain** — table / row / column / allowed values. **Degree** = number of columns; **cardinality** = number of rows. "Relational" comes from *relation* (a table), not "relationships".
- **Four table rules** — every row has the same columns, cells are atomic, rows have no order, no duplicate rows.
- **Primary key** identifies a row; **foreign key** stores another row's key; the DBMS enforces **referential integrity** (the referenced row must exist).
- **Declarative** — SQL says *what* you want; the optimizer decides *how*.
- **Schema-on-write** (relational: checked when written) vs **schema-on-read** (most NoSQL: checked by the app when read).
- **Polyglot persistence** — using several stores, each for the access pattern it suits.

Key facts & numbers

Component	Job
Query processor	**parser** (syntax + meaning) → **optimizer** (cheapest plan by estimated cost) → **executor**
Storage engine	fixed-size **pages** in files, indexes (InnoDB, WiredTiger, PG heap)
Buffer pool	caches hot pages in memory (PG `shared_buffers`, default 128MB)
Transaction + lock manager	ACID; **shared** locks (many readers) vs **exclusive** (one writer); deadlock detection
Recovery manager	**write-ahead log** (PG: WAL in `pg_wal/`): redo committed, undo uncommitted
Catalog	describes tables, columns, indexes, users (`pg_catalog`, `information_schema`)

Sub-language	Statements
DDL	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `COMMENT`
DML	`INSERT`, `UPDATE`, `DELETE`, `MERGE` (PG 15+), `COPY`
DQL	`SELECT`
DCL	`GRANT`, `REVOKE`, `CREATE ROLE`
TCL	`BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`, `RELEASE SAVEPOINT`

NoSQL family	Strength	Weakness	Examples
Document	read/write one whole varied object	limited cross-document joins	MongoDB, Couchbase
Key-value	extremely fast get/put by key	can't search inside values	Redis, DynamoDB
Wide-column	huge write throughput, partitioned	queries must follow partition key	Cassandra, HBase
Graph	multi-hop relationship queries	not for bulk aggregation	Neo4j, Neptune

ACID	BASE
Atomicity: all or nothing	**B**asically **A**vailable: answers even while nodes fail (maybe stale)
Consistency: declared rules still hold	**S**oft state: state changes as replicas catch up
Isolation: concurrent txns behave as if alone	**E**ventual consistency: replicas converge when writes stop
Durability: committed survives crashes	

Dialect	PostgreSQL	MySQL	SQLite	SQL Server
Limit	`LIMIT n` / `FETCH FIRST`	`LIMIT n`	`LIMIT n`	`TOP n` / `OFFSET… FETCH`
Concat	`||`, `concat()`	`concat()` (`||` = OR)	`||`	`+`, `concat()`
Auto-increment	`GENERATED … AS IDENTITY` / `serial`	`AUTO_INCREMENT`	`INTEGER PRIMARY KEY`	`IDENTITY(1,1)`
Transactional DDL	yes	no (implicit commit)	yes	mostly

Architectures: two-tier (client–server), three-tier (presentation / application / data), n-tier, cloud (IaaS: you run the DBMS on a VM; PaaS/DBaaS: RDS, Cloud SQL; SaaS), distributed (partitioning + replication). History: Codd 1970 → System R/SEQUEL, Oracle 1979 → ANSI SQL 1986, ISO 1987.

In the interview

Q: What are the parts of a DBMS? **A:** Query processor (parser, optimizer, executor), storage engine with pages and indexes, buffer pool, transaction and lock managers for ACID and concurrency, recovery manager with a write-ahead log, and a catalog describing everything.

Q: Why does the optimizer need statistics? **A:** To estimate how many rows each step returns, which decides index vs full scan and the join order.

Q: What is write-ahead logging? **A:** The log record of a change is flushed to disk before the changed data page, so after a crash the system can redo or undo from the log.

Q: Explain the relational model in 30 seconds. **A:** Codd (1970): data as relations of typed tuples, linked only by values (foreign keys the DBMS verifies). Rows are unordered, cells atomic; queries are declarative so the DBMS can optimise and reorganise freely.

Q: What happens when you delete a parent row that has children? **A:** Depends on the FK's `ON DELETE`: `NO ACTION`/`RESTRICT` (error, default), `CASCADE`, `SET NULL`, `SET DEFAULT`.

Q: Relational or NoSQL? **A:** Choose by access pattern: orders need multi-table transactions and constraints (relational); per-restaurant menus fetched as one unit suit a document store; sessions are key lookups (Redis). Relational DBs do scale horizontally (replicas, sharding, distributed SQL); the real cost is cross-node joins and transactions.

Q: Why not one database for everything? **A:** Often you should: every extra store adds operational cost and cross-store consistency problems; add one only when a requirement justifies it.

Gotchas

- Assuming SQL runs top to bottom as written → the optimizer may reorder joins and filters.
- Judging speed by a cold-cache first run → the second may come from the buffer pool.
- Confusing cache (memory, lost in a crash) with durability (the log on disk).
- Relying on row order without `ORDER BY` → it can change at any time.
- Lists in one cell (`'red,green,blue'`) → breaks atomicity; tables without a primary key → duplicates creep in.
- Unquoted identifiers fold to lower case in PostgreSQL; `"Employees"` keeps case. Strings use single quotes; escape `'` as `''`.
- In psql a typo inside a transaction aborts it → `ROLLBACK;`.

TL;DR

A DBMS (parser → optimizer → executor, storage engine, buffer pool, transaction/lock/recovery managers, catalog) mediates every access. Relational = typed tables linked by keys with enforced integrity; NoSQL families trade structure and joins for flexible shapes and scale-out; SQL splits into DDL, DML, DQL, DCL, TCL.

2. SELECT Basics

Reading, filtering, sorting and paging rows; NULL and three-valued logic; the logical processing order.

Must-know

- `SELECT` **list** — any expression can be a column; `AS alias` names it; aliases use double quotes (`"Total"`), strings single quotes.

- **`WHERE`** — keeps only rows whose condition is **`TRUE`** (not FALSE, not UNKNOWN).
- **Precedence** — `NOT` > `AND` > `OR`; parenthesise whenever `AND` and `OR` mix.
- **Integer division truncates** in PostgreSQL: `7/2 = 3`. `<>` is the standard not-equal. `||` concatenates.
- **`BETWEEN a AND b`** — inclusive on both ends; for timestamps use half-open `>= start AND < next_day`.
- **`IN (…)`** — shorthand for ORs. **`LIKE`** — `%` = any run of characters, `_` = exactly one; case-sensitive in PG. **`ILIKE`** — case-insensitive (PG). `~` / `~*` — regex.
- **`NULL`** — unknown. Any comparison with NULL gives **`UNKNOWN`** (three-valued logic). Test with `IS [NOT] NULL`; compare NULL-safely with `IS [NOT] DISTINCT FROM`.
- **`DISTINCT`** — removes duplicate result rows over the *whole* select list; NULLs count as equal for it.
- **`DISTINCT ON (key)`** (PG) — keeps the first row per key according to `ORDER BY` (which must start with the key).
- **`ORDER BY`** — the only guarantee of order; several keys each `ASC`/`DESC`; `NULLS FIRST/LAST`; add a unique **`tie-breaker`**.
- **`FETCH FIRST n ROWS WITH TIES`** — also returns rows tied with the last one.
- **Keyset pagination** — `WHERE (sort_key, id) > (:last_key, :last_id) ORDER BY sort_key, id LIMIT n`, backed by an index; deep `OFFSET` is O(offset) and unstable under inserts.
- **Logical order** — FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT. Aliases exist from SELECT onward, so they work in `ORDER BY` but not in `WHERE`.

Key facts & numbers

NULL case	Result
`NULL = NULL`	NULL (UNKNOWN), not TRUE
`WHERE x <> 5`	drops rows where `x` is NULL
`x NOT IN (1, NULL)`	never TRUE → returns no rows
`NULL IS DISTINCT FROM NULL`	FALSE
`''` is NULL?	no in PG, MySQL, SQL Server, SQLite; **yes in Oracle**
`count(col)`, `sum(col)`	skip NULLs

NULLs in ascending sort	Database
last (NULLS FIRST/LAST supported)	PostgreSQL, Oracle
first	MySQL, SQL Server, SQLite (SQLite supports NULLS FIRST/LAST from 3.30; MySQL/SQL Server use `ORDER BY col IS NULL, col` or `CASE`)

Paging	Syntax
PostgreSQL	`LIMIT 10 OFFSET 20` or `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
MySQL	`LIMIT 10 OFFSET 20` or `LIMIT 20, 10` (**offset first**)
SQL Server	`TOP (10)` or `ORDER BY … OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
Oracle 12c+	`OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`

`LIKE` case-sensitivity	**PG: sensitive (`ILIKE` for insensitive) · MySQL / SQL Server: collation (default insensitive) · SQLite: insensitive for ASCII**

Code to remember

```sql
-- first (latest) order per customer, PostgreSQL
SELECT DISTINCT ON (customer_id) customer_id, order_id, placed_at
FROM orders
ORDER BY customer_id, placed_at DESC, order_id DESC;

-- keyset "next page"
SELECT order_id, placed_at FROM orders
WHERE (placed_at, order_id) < (:last_ts, :last_id)
ORDER BY placed_at DESC, order_id DESC
LIMIT 20;
```

In the interview

Q: Explain NULL handling. **A:** NULL means unknown, so comparisons give UNKNOWN and `WHERE` keeps only TRUE. Use `IS NULL` / `IS DISTINCT FROM`; aggregates like `count(col)` and `sum` skip NULLs.

Q: Why can `NOT IN` return nothing? **A:** If the list or subquery contains a NULL, `x NOT IN (…)` is never TRUE. Use `NOT EXISTS` (or filter NULLs).

Q: How do you page through a large result? **A:** `ORDER BY` + `LIMIT/OFFSET` works but deep offsets read and discard rows and shift under inserts; keyset pagination on an indexed `(sort_key, id)` is constant-cost and stable.

Q: Top 3 salaries including ties? **A:** `FETCH FIRST 3 ROWS WITH TIES`, or `DENSE_RANK()` (Ch 8).

Q: Why can't I use a SELECT alias in WHERE? **A:** WHERE is evaluated before SELECT in the logical order; repeat the expression or wrap it in a subquery/CTE.

Q: SQL Server equivalent of LIMIT? **A:** `TOP (n)` or `OFFSET … FETCH`.

Gotchas

- `SELECT *` in application code → extra I/O, breaks when columns change; name columns.
- A missing comma (`SELECT first_name last_name`) silently makes an alias.
- `"Mumbai"` in double quotes is a column name, not a string.
- `= NULL` is never true → `IS NULL`.
- `LIKE 'user_%'` — `_` is a wildcard too.
- `DISTINCT` to hide duplicates from a bad join → fix the join (fan-out); need counts → `GROUP BY`.
- `LIMIT` without `ORDER BY` → arbitrary rows; paging on a non-unique key → rows repeat or vanish.
- Numbers stored as text sort alphabetically (`'10' < '9'`).

TL;DR

WHERE keeps TRUE rows; NULL is unknown (use IS NULL, IS DISTINCT FROM, avoid NOT IN with NULLs). Only ORDER BY orders; add tie-breakers; prefer keyset over deep OFFSET. Logical order FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT.

3. Functions and Expressions

Types and casts, number, string and date/time functions, time zones, and the conditional expressions `CASE`, `COALESCE`, `NULLIF`, `GREATEST`/`LEAST`.

Must-know

- **Casting** — `CAST(x AS t)` (standard, everywhere), `x::t` (PostgreSQL), `date '2025-03-18'` (typed literal). PostgreSQL refuses silent coercions — a feature.

- **`numeric(p,s)`** — exact decimal; use for **money**. `double precision` is approximate: `0.1 + 0.2 = 0.30000000000000004`.
- **`::integer` rounds**, it does not truncate; use `trunc()` to cut.
- **Rounding** — `round` (numeric: half away from zero; floats: half to even), `trunc`, `ceil` (up: `ceil(-4.3) = -4`), `floor` (down: `floor(-4.3) = -5`).
- **`log(x)` is base 10 in PostgreSQL**; natural log is `ln(x)`. (MySQL `LOG(x)` = natural log.)
- **Concatenation and NULL** — `a || NULL` is NULL; `concat()` skips NULLs; `concat_ws(sep, …)` joins with a separator skipping NULLs.
- **String positions start at 1**; `strpos` returns 0 when absent. `length` = characters, `octet_length` = bytes.
- **`timestamptz`** — an absolute instant stored in UTC, shown in the session zone. **`timestamp`** — zone-less wall-clock reading. Store instants as `timestamptz`.
- **`now()`** = transaction start (fixed within a transaction); **`clock_timestamp()`** changes every call.
- **`date_trunc('month', ts)`** truncates to a period; **`extract(field FROM ts)`** pulls a field; `to_char`/`to_date` format and parse (display only).
- **Month arithmetic clamps** — `Jan 31 + 1 month = Feb 28`, so "+1 month −1 month" may not return the start.
- **`CASE`** — first TRUE branch wins; no `ELSE` → NULL. **`COALESCE`** — first non-NULL (defaults). **`NULLIF(a,b)`** — NULL when equal (safe division `x / NULLIF(y,0)`). **`GREATEST`**/**`LEAST`** — PG ignores NULLs; MySQL/Oracle return NULL if any is NULL.

Key facts & numbers

Type	Notes
`smallint`/`integer`/`bigint`	2/4/8 bytes; `integer` max 2,147,483,647
`numeric(p,s)`	exact; rounded to *s* decimals when stored (`12.345::numeric(5,2)` → 12.35)
`text` vs `varchar(n)`	same performance in PG; `char(n)` pads with spaces
`interval`	`3 days`, `1 mon`, `02:30:00`

Date arithmetic	Result
`date + integer`	date (add days)
`date - date`	integer days
`timestamp ± interval`	timestamp
`timestamp - timestamp`	interval
`age(later, earlier)`	"years mons days"
hours between	`extract(EPOCH FROM end - start) / 3600`

Task	MySQL	PostgreSQL	SQL Server
Add 3 days	`DATE_ADD(d, INTERVAL 3 DAY)`	`d + interval '3 days'`	`DATEADD(day, 3, d)`
Days between	`DATEDIFF(end, start)`	`end::date - start::date`	`DATEDIFF(day, start, end)`
Truncate to month	`DATE_FORMAT(d,'%Y-%m-01')`	`date_trunc('month', d)`	`DATETRUNC(month, d)` (2022+)
First non-NULL	`IFNULL`, `COALESCE`	`COALESCE`	`ISNULL`, `COALESCE`
If/else	`IF(c,a,b)`	`CASE`	`IIF(c,a,b)`
Split field	`SUBSTRING_INDEX`	`split_part`	`STRING_SPLIT`
Characters / bytes	`CHAR_LENGTH` / `LENGTH`	`length` / `octet_length`	`LEN` / `DATALENGTH`
Power	`POWER`	`power`, `^`	`POWER` (`^` is XOR!)

AT TIME ZONE	Gives
`timestamptz AT TIME ZONE 'Zone'`	wall-clock `timestamp` in that zone
`timestamp AT TIME ZONE 'Zone'`	the `timestamptz` instant when that zone showed that reading

Code to remember

```sql
SELECT order_id,
       CASE WHEN total >= 1000 THEN 'large'
            WHEN total >= 100  THEN 'medium'
            ELSE 'small' END                    AS size,
       COALESCE(email, 'no email')              AS contact,
       round(100.0 * returned / NULLIF(sold, 0), 1) AS return_pct,
       date_trunc('month', placed_at)::date     AS month
FROM ...;
```

In the interview

Q: How do you handle dates in reports? **A:** Store instants as `timestamptz`, group with `date_trunc`, filter with half-open ranges so indexes still work, compute durations by subtraction or `EXTRACT(EPOCH …)`; remember month arithmetic clamps and `now()` is fixed per transaction.

Q: `timestamp` vs `timestamptz`? **A:** `timestamptz` is an absolute instant converted to and from the session zone; `timestamp` is a zone-less wall-clock reading.

Q: Orders per month including months with zero orders? **A:** Generate months with `generate_series` and LEFT JOIN the counts, `COALESCE(count, 0)`.

Q: What are CASE, COALESCE, NULLIF for? **A:** CASE is if/else anywhere an expression goes (including inside SUM); COALESCE gives defaults and stops NULL wiping arithmetic; NULLIF avoids division by zero.

Q: `ISNULL` vs `COALESCE` in SQL Server? **A:** `ISNULL` takes two arguments and the first one's type; `COALESCE` is standard, takes many, uses type precedence.

Q: Why not store money as float? **A:** Binary floats can't represent most decimal fractions exactly, so sums drift; `numeric` is exact.

Gotchas

- Numbers or dates in text columns → alphabetical sorting.
- `WHERE date_trunc('month', placed_at) = …` or other functions on an indexed column → index unusable; rewrite as a range.
- Local times in `timestamp` for a multi-zone app → use `timestamptz`.
- `CASE x WHEN NULL` is never true; mixed branch types (`THEN 1 ELSE 'none'`) error in PG.
- Overlapping `WHEN`s in the wrong order → first match wins.
- Formatting (`to_char`) before sorting or calculating → sorts text.

TL;DR

`numeric` for money, `timestamptz` for instants, cast with `CAST`/`::`. PG `log` is base 10; `||` with NULL gives NULL; `now()` is per transaction; month maths clamps. `CASE`, `COALESCE`, `NULLIF`, `GREATEST`/`LEAST` handle conditions and NULLs.

4. Aggregation

Aggregate functions and NULLs, `GROUP BY`, `HAVING`, conditional aggregation, the logical query order, and `ROLLUP`/`CUBE`/`GROUPING SETS`.

Must-know

- **`COUNT(*)`** counts rows; **`COUNT(col)`** counts non-NULL values; **`COUNT(DISTINCT col)`** counts different non-NULL values. `COUNT(1)` = `COUNT(*)`.
- **NULLs skipped** — every aggregate except `COUNT(*)` ignores NULL inputs; `AVG` divides by `COUNT(col)`, not `COUNT(*)`.
- **Empty input** — `COUNT` → 0, every other aggregate → **NULL** (not 0).
- **No `GROUP BY`** → the whole filtered table is one group → exactly one row.
- **`GROUP BY`** — one output row per distinct key; select only keys and aggregates (PG also allows columns functionally dependent on a grouped primary key). NULL keys form **one** group. No guaranteed order.
- **`HAVING`** — filters groups after aggregation; aggregates allowed. **`WHERE`** filters rows before grouping; no aggregates. Put row filters in `WHERE` (cheaper).
- **Conditional aggregation** — `count(*) FILTER (WHERE c)` (PG, SQLite 3.30+) or portable `SUM(CASE WHEN c THEN 1 ELSE 0 END)` / `COUNT(CASE WHEN c THEN 1 END)`. Many filtered numbers in one scan; powers breakdowns, ratios and fixed pivots.
- **Ratios** — multiply by `100.0` first to avoid integer division.
- **`ROLLUP(a,b)`** → `(a,b)`, `(a)`, `()`; **`CUBE(a,b)`** → all 2^n subsets; **`GROUPING SETS`** → exactly what you list. **`GROUPING(col)`** = 1 on rolled-up rows (tells subtotal NULL from real NULL).
- **Hash vs sort aggregation** — hash table of groups (fast, needs memory) vs sort + scan runs (can use an index order, streams).
- **Need each row *and* its group total?** → window function `avg(salary) OVER (PARTITION BY dept_id)`, not `GROUP BY`.

Key facts & numbers

Step	Clause	Can use
1	FROM / JOIN … ON	table columns
2	WHERE	columns; no aggregates, no SELECT aliases
3–4	GROUP BY + aggregates	columns / expressions
5	HAVING	grouping columns, aggregates
6	SELECT (window functions run here)	grouping columns, aggregates
7	DISTINCT	output columns
8	UNION / INTERSECT / EXCEPT	output columns
9	ORDER BY	aliases, positions, input columns
10	LIMIT / OFFSET	—

Mnemonic: **F**rom → **W**here → **G**roup → **H**aving → **S**elect → **D**istinct → **O**rder → **L**imit.

PostgreSQL extras	Use
`string_agg(x, ',' ORDER BY …)`	list values per group (MySQL `GROUP_CONCAT`, SQL Server `STRING_AGG`)
`array_agg`, `bool_and` / `bool_or`	array; "all/any true"
`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	median (interpolated); `percentile_disc` = an actual value
`mode() WITHIN GROUP (ORDER BY x)`	most frequent value

Non-grouped column in SELECT	Behaviour
PostgreSQL	error unless dependent on a grouped PK
MySQL 8	error (`ONLY_FULL_GROUP_BY`, default since 5.7.5); off → arbitrary value
SQLite	allowed, arbitrary row ("bare column")
SQL Server, Oracle	always an error

Alias allowed in	GROUP BY	HAVING	WHERE
PostgreSQL	yes	no	no
MySQL	yes	yes	no
SQL Server	no	no	no

Subtotal support: PG 9.5+, SQL Server, Oracle: all; MySQL 8: `WITH ROLLUP` + `GROUPING()` only; SQLite: none (`UNION ALL`). SQL Server `AVG(int)` returns an **int** — use `AVG(1.0 * x)`.

Code to remember

```sql
SELECT date_trunc('month', placed_at)::date AS month,
       count(*)                                      AS orders,
       count(*) FILTER (WHERE status = 'cancelled')  AS cancelled,
       round(100.0 * count(*) FILTER (WHERE status = 'cancelled')
             / count(*), 1)                          AS cancel_pct
FROM orders
GROUP BY 1
HAVING count(*) >= 2
ORDER BY 1;

-- duplicates
SELECT lower(email), count(*) FROM customers
GROUP BY lower(email) HAVING count(*) > 1;
```

In the interview

Q: `COUNT(*)` vs `COUNT(1)` vs `COUNT(col)`? **A:** The first two count rows (identical); `COUNT(col)` counts non-NULL values of `col`.

Q: `AVG(x)` vs `SUM(x)/COUNT(*)`? **A:** They differ when `x` has NULLs (AVG divides by `COUNT(x)`), and `SUM(int)/COUNT(*)` is integer division in PG.

Q: WHERE vs HAVING? **A:** WHERE filters rows before grouping, no aggregates; HAVING filters groups after, aggregates allowed. Use WHERE whenever no aggregate is needed.

Q: Departments with more than 3 employees? **A:** `SELECT dept_id FROM employees GROUP BY dept_id HAVING count(*) > 3;`

Q: Why can't I filter on `ROW_NUMBER()` in WHERE? **A:** Window functions run in the SELECT step, after WHERE; filter in an outer query or CTE.

Q: Does GROUP BY sort? **A:** No; hash aggregation returns groups in no order. Add ORDER BY.

Q: ROLLUP vs CUBE? How many groupings in `CUBE(a,b,c)`? **A:** ROLLUP = hierarchical subtotals plus grand total; CUBE = every combination; $2^3 = 8$.

Q: Median in PostgreSQL? **A:** `percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`.

Gotchas

- `COUNT(CASE WHEN c THEN 1 ELSE 0 END)` counts every row (0 is not NULL) → use `SUM` with `ELSE 0` or `COUNT` without `ELSE`.
- `SUM` of nothing is NULL in app code → `COALESCE(sum(x), 0)`.
- "Fixing" a non-grouped column by adding it to GROUP BY → silently smaller groups. Ask "one row per *what*?".
- Groups only come from existing rows (an empty department has no group) → LEFT JOIN from the dimension.
- `HAVING headcount > 2` using a SELECT alias fails in PG → repeat the aggregate.
- Putting a condition in WHERE filters rows for *all* aggregates → use FILTER for one.
- `COUNT(DISTINCT a, b)` not accepted in PG → `COUNT(DISTINCT (a, b))`.
- Subtotal NULLs mistaken for data NULLs → `GROUPING()`.

TL;DR

Aggregates skip NULLs (except `COUNT(*)`); empty input gives 0 for COUNT, NULL for the rest. GROUP BY → one row per key; HAVING filters groups; FILTER/CASE for conditional counts. Logical order FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

5. Joins

Cartesian product, inner/left/right/full/self joins, semi- and anti-joins, join pitfalls, and how joins execute.

Must-know

- **Every join is a filtered Cartesian product** — `CROSS JOIN` gives m × n pairs; `ON` keeps the pairs that satisfy the condition.
- **Explicit `JOIN … ON`** (SQL-92) over comma joins: condition sits by the join; required for outer joins.
- **Equi-join** uses only `=`; **non-equi join** uses `<`, `BETWEEN` … (e.g. price bands).
- **INNER** — matched pairs only; unmatched and NULL-keyed rows vanish; one-to-many multiplies rows.
- **LEFT** — every left row, right columns NULL-padded where no match. Includes zeros; with `WHERE right.pk IS NULL` finds "missing".
- **RIGHT** = LEFT with tables swapped (write it as LEFT). **FULL** — everything from both sides; used for **reconciliation**.
- **Self join** — one table, two aliases, two roles: manager hierarchies, pairs (`a.id < b.id`), row-to-row comparisons. Arbitrary depth needs a recursive CTE.
- **Semi-join** — rows of A with ≥ 1 match in B, once each: `EXISTS` / `IN`. **Anti-join** — rows with no match: `NOT EXISTS` or `LEFT JOIN … WHERE b.pk IS NULL`.
- **`ON` vs `WHERE` in a LEFT JOIN** — `ON` decides which right rows match (left rows kept); `WHERE` on a right column removes padded rows → becomes an INNER join.
- **Fan-out** — joining a parent to two child tables multiplies rows (4 employees × 2 projects = 8), doubling sums. Fix: aggregate each child separately, then join.
- **NULL keys never match** (`NULL = NULL` is UNKNOWN); `IS NOT DISTINCT FROM` matches them but blocks hash/merge joins.
- **`USING (col)`** merges the same-named key column; **`NATURAL JOIN`** joins on *all* same-named columns → breaks when a column is added. Avoid.

Key facts & numbers

Join	Returns	Typical use
CROSS	every pair (m × n)	grids, combinations
INNER	matched pairs	orders with customers
LEFT	all left + matches	all customers, with orders if any
FULL	all rows of both	reconciliation
Semi (`EXISTS`)	left rows with a match, once	customers who ordered
Anti (`NOT EXISTS`)	left rows with no match	customers who never ordered

Anti-join form	NULL-safe?	Note
`NOT EXISTS`	yes	clearest; planned as anti join — **recommended**
`LEFT JOIN … WHERE b.pk IS NULL`	yes, if testing a never-NULL column	also an anti join; gives access to other columns
`NOT IN (subquery)`	**no**	one NULL in the subquery → empty result

Algorithm	How	Good when
Nested loop	per outer row, look up inner (ideally via index)	small outer + index on inner key; only option for non-equi
Hash join	build hash on smaller input, probe with the other	large unsorted equi-joins; O(n+m), needs memory
Merge join	sort both (or index order), walk together	inputs already sorted / huge

Feature	PG	MySQL 8	SQLite	SQL Server
FULL JOIN	yes	**no** (LEFT UNION RIGHT)	3.39+	yes
USING / NATURAL	yes	yes	yes	no
LATERAL	yes	8.0.14+	no	`CROSS/OUTER APPLY`
NULL-safe equal	`IS NOT DISTINCT FROM`	`<=>`	`IS`	2022+

Code to remember

```sql
-- every customer with delivered-order count, zeros kept
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
                  AND o.status = 'delivered'     -- right-table filter in ON
GROUP BY c.customer_id;

-- employees earning more than their manager
SELECT e.first_name FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;
```

In the interview

Q: Explain the join types. **A:** Inner keeps matches; left/right keep all rows of one side NULL-padded; full keeps both; cross gives every combination; self joins use two aliases on one table.

Q: WHERE vs ON in a left join? **A:** ON decides matches; WHERE filters the joined result and drops padded rows, turning it into an inner join.

Q: Why did my row count go up after a join? **A:** One-to-many: each left row repeats per matching right row. Joining two child tables causes fan-out; aggregate each first.

Q: Customers who never ordered? **A:** `NOT EXISTS` (preferred), `LEFT JOIN … IS NULL`, or `NOT IN` only if the column is non-nullable.

Q: Why not JOIN + DISTINCT for a semi-join? **A:** It multiplies then de-duplicates; `EXISTS` stops at the first match and states intent.

Q: Full outer join in MySQL? **A:** `LEFT JOIN … UNION … RIGHT JOIN` (or UNION ALL with the right half filtered to `a.key IS NULL`).

Q: Rows in exactly one of two tables? **A:** `FULL JOIN … WHERE a.pk IS NULL OR b.pk IS NULL`, or `EXCEPT` both ways.

Q: Which join algorithms exist? **A:** Nested loop, hash, merge; the optimizer picks by sizes, indexes and sort order. Index your foreign keys.

Gotchas

- `count(*)` after a LEFT JOIN counts the padded row (empty dept shows 1) → `count(right.pk)`.

- `a LEFT JOIN b … JOIN c ON c.x = b.x` drops the padded rows again → make the second join LEFT too.
- Inner join for a hierarchy loses the root (the CEO) → LEFT JOIN.
- Pairs listed twice → `a.id < b.id`.
- `count(DISTINCT …)` hides fan-out for counts but **not** for sums.
- Joining on the wrong same-typed columns "works" and returns nonsense; casting inside `ON` blocks index use.
- A "one" side that isn't unique fans out → check `GROUP BY key HAVING count(*) > 1`.

TL;DR

Inner = matches, left = keep left, full = keep both, cross = all pairs. Semi-join `EXISTS`, anti-join `NOT EXISTS`. Watch fan-out, right-table filters in WHERE, NULL keys and `count(*)` on padded rows; never NATURAL JOIN.

6. Subqueries and Set Operations

Scalar, `IN`, correlated and `EXISTS` subqueries, `ANY` / `ALL`, derived tables, `LATERAL`, and `UNION` / `INTERSECT` / `EXCEPT`.

Must-know

- **Subquery kinds by result** — scalar (one value), row (one row, several columns), column (one column, many rows: `IN` / `ANY` / `ALL`), table (`FROM`, `EXISTS`).
- **Non-correlated** runs conceptually once; **correlated** references the outer row, conceptually once per outer row.
- **Scalar subquery edge cases** — 0 rows → NULL; 2+ rows → run-time error.
- **`NOT IN` trap** — a NULL in the list makes `x NOT IN (…)` UNKNOWN for every non-matching `x` → empty result. `IN` has no such problem.
- **`EXISTS`** — "is there at least one row?"; never UNKNOWN, ignores the select list, stops at the first match. **`NOT EXISTS`** is the NULL-safe anti-join; **double `NOT EXISTS`** expresses "for all" (relational division).
- **IN vs EXISTS performance** — modern optimizers (PG) turn both into the same semi-join; choose by clarity, but always `NOT EXISTS` over `NOT IN`.
- **`ANY` / `SOME`** — compare with at least one; **`ALL`** — with every one. `= ANY` ≡ `IN`; `<> ALL` ≡ `NOT IN` (same NULL trap); `ALL` over empty = TRUE.
- **Derived table** — subquery in `FROM`: build in stages, aggregate an aggregate, pre-aggregate before a join (needs an alias before PG 16).
- **`LATERAL`** — derived table that can see earlier `FROM` items, evaluated per outer row; top-N / latest row per group. `LEFT JOIN LATERAL (…) ON true` keeps outer rows with no result. SQL Server: `CROSS/OUTER APPLY`.
- **Set operations** — same column count, compatible types, names from the first query, **NULLs treated as equal**. `UNION` de-duplicates (sort/hash cost), **`UNION ALL`** doesn't — default to it. `INTERSECT` = in both; `EXCEPT` = first minus second (not commutative).
- **Precedence** — `INTERSECT` binds tighter than `UNION` / `EXCEPT` (left to right); use parentheses. Final `ORDER BY` / `LIMIT` apply to the whole result.

Key facts & numbers

`NOT IN` truth table	list `(1,2)`	list `(1,2,NULL)`
`3 IN (list)`	FALSE	UNKNOWN
`3 NOT IN (list)`	TRUE	**UNKNOWN (row dropped)**
`1 NOT IN (list)`	FALSE	FALSE

Block list `(4, NULL)` over 10 customers: `IN` 1, `EXISTS` 1, **`NOT IN` 0**, `NOT EXISTS` 9.

Bag semantics (row *m* times in A, *n* in B)	Result
`UNION ALL`	m + n
`INTERSECT ALL`	min(m, n)
`EXCEPT ALL`	max(m − n, 0)
`UNION` / `INTERSECT` / `EXCEPT`	once (if present per the rule)

Support	PG	MySQL	SQLite	SQL Server	Oracle
INTERSECT / EXCEPT	yes	8.0.31+	yes	yes	`MINUS` (+EXCEPT 21c)
INTERSECT/EXCEPT ALL	yes	8.0.31+	no	no	21c+
ANY/ALL with subquery	yes	yes	**no**	yes	yes

You want…	Best tool
Columns from related tables	JOIN
Has / has no match	EXISTS / NOT EXISTS
Compare with one computed value	scalar subquery
Build in stages, pre-aggregate	derived table / CTE
Top-N / latest per group	LATERAL or window function
Stack results	UNION ALL
Whole-row, NULL-safe diff	INTERSECT / EXCEPT

Code to remember

```sql
-- above their own department's average (correlated)
SELECT e.first_name, e.salary FROM employees e
WHERE e.salary > (SELECT avg(x.salary) FROM employees x WHERE x.dept_id = e.dept_id);

-- latest 2 orders per customer
SELECT c.customer_id, o.order_id, o.placed_at
FROM customers c
LEFT JOIN LATERAL (SELECT order_id, placed_at FROM orders o
                   WHERE o.customer_id = c.customer_id
                   ORDER BY placed_at DESC LIMIT 2) o ON true;

-- are two queries identical?  must be empty
(q1 EXCEPT ALL q2) UNION ALL (q2 EXCEPT ALL q1);
```

In the interview

Q: Why does NOT IN return no rows? **A:** The list contains a NULL; `x <> NULL` is UNKNOWN so the AND chain is never TRUE. Use NOT EXISTS.

Q: IN vs EXISTS? **A:** IN asks "is this value in the list", EXISTS "is there a matching row"; both become semi-joins in PG. Negated, only NOT EXISTS is NULL-safe.

Q: Rewrite "above department average" without a correlated subquery. **A:** Join to a derived table of department averages, or compute `avg(salary) OVER (PARTITION BY dept_id)` in a subquery and filter outside.

Q: When is a subquery better than a join? **A:** Existence tests, comparing with an aggregate, or aggregating before joining; a join is better when you need columns from both sides.

Q: UNION vs UNION ALL? **A:** UNION removes duplicates (within each input too) at the cost of a sort/hash; UNION ALL just appends.

Q: IDs in A but not in B? **A:** `EXCEPT` (returns distinct compared columns only) or `NOT EXISTS` (returns full rows of A).

Q: How do you get the whole of "customers who bought every product"? **A:** Double NOT EXISTS, or `GROUP BY customer HAVING count(DISTINCT product) = (SELECT count(*) FROM products)`.

Gotchas

- `=` with a subquery that can return several rows → use `IN`.
- `ORDER BY` inside a subquery doesn't order the outer result.
- Uncorrelated by accident: `WHERE x.dept_id = dept_id` resolves to `x.dept_id = x.dept_id` → always qualify with the outer alias.
- Per-row scalar subqueries on big tables without an index on the correlated column → slow; use joins/windows.
- `x <> ANY (list)` means "differs from at least one" — rarely intended.
- `ORDER BY` in a UNION branch only works inside parentheses (usually with LIMIT).

TL;DR

Scalar subqueries: 0 rows → NULL, 2+ → error. Use EXISTS/NOT EXISTS for semi/anti-joins, never NOT IN on nullable columns. Derived tables and LATERAL build in stages. UNION ALL by default; INTERSECT/EXCEPT for NULL-safe whole-row diffs.

7. CTEs, Views and Recursion

`WITH` queries, data-modifying CTEs, recursive CTEs for trees and graphs, `generate_series`, views and materialized views.

Must-know

- **CTE** — `WITH name AS (query)` names a step for **one statement**; chain with commas under one `WITH`; a CTE sees only those above it.
- **Inlining** — PostgreSQL 12+ inlines a CTE referenced once (same cost as a subquery); `AS MATERIALIZED` / `NOT MATERIALIZED` override. A CTE is not a cache.
- **Data-modifying CTE** (PG only) — `INSERT/UPDATE/DELETE … RETURNING` inside `WITH`; the whole statement is atomic and every part sees the **pre-statement snapshot** (read changes from `RETURNING`, not the table).
- **Recursive CTE** — anchor `UNION [ALL]` recursive term. The recursive term sees **only the previous pass's rows** (working table); stops when a pass returns nothing.

- **Anchor fixes column types** — cast there (`first_name::text`, `1::numeric`) or recursion errors/overflows.
- **Trees (adjacency list)** — down: join children `e.manager_id = t.emp_id`; up: join parent `m.emp_id = t.manager_id`; subtree totals: carry the top node, aggregate outside. Carry depth and a path for ordering.
- **Cycles** — carry a visited array and add `WHERE NOT id = ANY(path)`, or PG 14's `CYCLE` clause; `SEARCH BREADTH/DEPTH FIRST` gives an ordering column.
- **`generate_series`** — rows of numbers/dates; full calendar + LEFT JOIN + `count(col)` shows empty periods as 0. MySQL: recursive CTE or a calendar table.
- **View** — stored query, no data, always current, same speed as its query. For simplicity, security (hide columns/rows), stable interface, one metric definition.
- **Updatable view** — simple single-table views are auto-updatable; **`WITH CHECK OPTION`** blocks writes that would fall outside the view.
- **Materialized view** — stored result; indexable; stale until `REFRESH MATERIALIZED VIEW`; `CONCURRENTLY` avoids blocking readers but needs a **unique index**. PG has **no auto/incremental refresh**.

Key facts & numbers

Need	Use	Lifetime	Indexable
One-off intermediate, used once	subquery	statement	no
Readable steps, reuse, recursion, atomic moves	CTE	statement	no
Reusable definition, security, stable interface	view	permanent	no (index base tables)
Fast reads of expensive, slightly stale results	materialized view	permanent	yes
Intermediate reused by many statements	`TEMP TABLE`	session / txn	yes

Recursion	PG	MySQL 8	SQL Server
Keyword	`WITH RECURSIVE`	`WITH RECURSIVE`	plain `WITH`
Depth limit	none	`cte_max_recursion_depth` 1000	100 (`OPTION (MAXRECURSION n)`, 0 = unlimited)
SEARCH / CYCLE	14+	no	no

Tree model	Reads	Writes
Adjacency list (`parent_id`)	recursive CTE	trivial — default
Materialized path (`'1/2/15'`, `ltree`)	`LIKE '1/2/%'`, indexable	moving a subtree rewrites paths
Nested sets (`lft`, `rgt`)	subtree = range	inserts renumber many rows
Closure table (ancestor, descendant, depth)	simple joins	many rows per insert

Materialized views elsewhere: Oracle has incremental "fast refresh"; SQL Server "indexed views" update on every write; MySQL has none.

Code to remember

```sql
WITH RECURSIVE org AS (
  SELECT emp_id, first_name::text AS path, 0 AS depth
  FROM employees WHERE manager_id IS NULL             -- anchor
  UNION ALL
  SELECT e.emp_id, org.path || ' > ' || e.first_name, org.depth + 1
  FROM employees e JOIN org ON e.manager_id = org.emp_id
)
SELECT * FROM org ORDER BY path;

-- zero-filled daily counts
SELECT d::date, count(o.order_id)
FROM generate_series('2025-01-01'::date, '2025-01-31', '1 day') d
LEFT JOIN orders o ON o.placed_at >= d AND o.placed_at < d + interval '1 day'
GROUP BY d ORDER BY d;
```

In the interview

Q: What is a CTE and is it faster than a subquery? **A:** A named subquery for one statement; usually identical cost because PG 12+ inlines it. Materialized, it can be slower (outer filters not pushed in) or faster (expensive step computed once).

Q: CTE vs temp table? **A:** CTE lasts one statement and has no indexes; a temp table lasts the session, can be indexed and analyzed, and is reusable.

Q: How does a recursive CTE work? **A:** Run the anchor, then repeatedly apply the recursive term to the previous batch, appending results, until a batch is empty.

Q: Stopping infinite loops? **A:** Visited-path array with `NOT id = ANY(path)`, or the `CYCLE` clause; MySQL/SQL Server also have depth limits.

Q: Huge tree queried constantly? **A:** Index `manager_id`; consider a closure table or materialized path (`ltree` + GiST).

Q: Does a view improve performance? **A:** No, it is expanded into the query; a materialized view (or SQL Server indexed view) can.

Q: Can you insert into a view? **A:** Yes if simple single-table (auto-updatable) or with `INSTEAD OF` triggers.

Q: Keeping a materialized view fresh? **A:** Scheduled refresh (pg_cron) or after each ETL batch, `CONCURRENTLY` with a unique index; for real time maintain a summary table incrementally.

Gotchas

- `WITH a AS (…) WITH b AS (…)` → one `WITH`, commas.
- Recursive CTE with no stop condition → runs until resources run out.
- Output order of recursion isn't guaranteed → `ORDER BY` path/depth.
- Anchoring on `manager_id = X` excludes X → anchor on `emp_id = X`.
- `SELECT *` in a view is expanded at creation → new table columns don't appear.
- PG `CREATE OR REPLACE VIEW` can only append columns.
- Plain `REFRESH` blocks readers; forgetting to schedule refreshes → stale dashboards.
- Deep view stacks → huge, untunable queries.

TL;DR

CTEs name steps (inlined in PG 12+); recursive CTEs walk trees and graphs (anchor + recursive term, stop condition, cast in anchor, carry path). `generate_series` fills gaps. Views are stored queries; materialized views are stored results refreshed by hand.

8. Window Functions

`OVER`, `PARTITION BY`, frames, ranking, `lag`/`lead`, running totals, top-N per group, and gaps and islands — the heart of SQL interviews.

Must-know

- **Window function** — `f(…) OVER (PARTITION BY … ORDER BY … frame)`: computed per row across related rows; **row count never changes** (unlike `GROUP BY`).
- `OVER ()` = all rows; `PARTITION BY` = groups without collapsing (share of total, difference from group average).
- `ORDER BY` **inside** `OVER` turns aggregates into running aggregates; it does not order the output (the final `ORDER BY` does).
- **Default frame with ORDER BY** = `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` → includes **peers (ties)**, so running totals "jump" on duplicate keys. Use a unique order or `ROWS`.
- **Frame modes** — `ROWS` = physical rows; `RANGE` = value distance (ties together, intervals allowed); `GROUPS` = peer groups; `EXCLUDE` removes rows (PG 11+).
- **Ranking** — `row_number` (unique, arbitrary among ties), `rank` (ties share, gaps: 1,2,2,4), `dense_rank` (no gaps: 1,2,2,3), `ntile(n)`, `percent_rank`, `cume_dist`.
- `lag`/`lead(expr, offset, default)` — previous/next row per partition; ignore the frame. `first_value`/`last_value`/`nth_value` use the frame → `last_value` needs `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.
- **Windows can't go in** `WHERE` — they run in the SELECT step; rank in a CTE/subquery, filter outside. No `QUALIFY` in PG/MySQL/SQL Server (Snowflake, BigQuery, DuckDB have it).
- **Gaps and islands** — consecutive values: `value − row_number()` is constant within a run → group by it. General rule: flag starts with `lag`, running `sum` of flags = island id. Gaps: `lead(v) − v > 1`.
- **Windows on GROUP BY** — windows run after grouping, so `sum(sum(x)) OVER ()` works.
- **Performance** — one sort per distinct window; an index on (partition, order) can remove it; `WINDOW w AS (…)` names a reused window.

Key facts & numbers

Ranking question	Use
Exactly one row per group / dedup / paging	`row_number()` + unique tie-breaker
Places with ties, next place skips (sports)	`rank()`
Nth highest *distinct* value, top N salary levels	`dense_rank()`
Top 10 %	`percent_rank() <= 0.1` or `cume_dist() <= 0.1`
Split into 4 equal groups	`ntile(4)`

Top-N filter	Rows per group
`row_number() <= N`	exactly N
`rank() <= N`	N places, can exceed N
`dense_rank() <= N`	N distinct values, can far exceed N

Metric	Window
Running total	`sum(x) OVER (PARTITION BY g ORDER BY t ROWS UNBOUNDED PRECEDING)`
Moving avg, last 3 rows	`avg(x) OVER (… ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Last 7 **days** with gaps	`RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW`
Share of total	`x / sum(x) OVER (PARTITION BY g)`
Growth	`x / lag(x) OVER (ORDER BY t) - 1`

Frame support	PG	MySQL 8	SQLite	SQL Server
Since	8.4	8.0	3.25	2005 (ranking) / 2012 (frames)
RANGE with offsets	11+	yes	yes	only UNBOUNDED/CURRENT ROW
GROUPS, EXCLUDE	11+	no	3.28+	no

`IGNORE NULLS` for `lag`/`first_value` only from PostgreSQL 19.

Code to remember

```sql
-- top 3 distinct salaries per department
WITH r AS (
  SELECT dept_id, first_name, salary,
         dense_rank() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dr
  FROM employees)
SELECT * FROM r WHERE dr <= 3;

-- login streaks (islands)
WITH d AS (SELECT DISTINCT user_id, login_date FROM logins),
g AS (SELECT user_id, login_date,
             login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
      FROM d)
SELECT user_id, min(login_date), max(login_date), count(*) AS days
FROM g GROUP BY user_id, grp HAVING count(*) >= 3;
```

In the interview

Q: Window function vs GROUP BY? **A:** GROUP BY collapses rows to one per group; a window computes across related rows but keeps every row.

Q: ROW_NUMBER vs RANK vs DENSE_RANK? **A:** Unique numbers; ties share with gaps; ties share without gaps. Pick by how ties should count.

Q: Top N per group? **A:** Rank in a CTE partitioned by group ordered by the measure, filter `<= N` outside; index `(group, measure DESC)`; for few groups `LATERAL … LIMIT N`; PG top-1 shortcut `DISTINCT ON`.

Q: Why does my running total repeat values? **A:** Default RANGE frame includes ties; order by a unique key or use `ROWS`.

Q: ROWS vs RANGE vs GROUPS? **A:** Physical rows vs value distance (ties included, intervals) vs groups of tied rows.

Q: Why does LAST_VALUE return the current row? **A:** The default frame ends at the current row; specify a full frame.

Q: Year-over-year growth per product? **A:** `lag(revenue) OVER (PARTITION BY product ORDER BY year)`, then `(revenue − prev) / prev`.

Q: Users with 3+ consecutive login days? **A:** De-duplicate per day, islands via `date − row_number()`, `HAVING count(*) >= 3`.

Q: Before window functions? **A:** Self-join on `t2.day = t1.day − 1` (breaks on gaps) or correlated "previous row" subquery.

Gotchas

- Expecting a window to reduce rows → use GROUP BY or filter outside.
- `ROWS 6 PRECEDING` for "last 7 days" when days are missing → `RANGE` with an interval (or fill the calendar).
- `lag` without `PARTITION BY` crosses groups; `lag` as "previous period" when periods can be missing.
- First row's `x − lag(x)` is NULL → pass a default or `coalesce`.
- Frame is ignored by `row_number`, `rank`, `lag`, `lead`, `ntile`.
- `row_number` with ties and no tie-breaker → non-deterministic results.
- Porting GROUPS/EXCLUDE/RANGE offsets to SQL Server or MySQL → unsupported.

TL;DR

`f() OVER (PARTITION BY … ORDER BY … frame)` keeps every row. Mind the default RANGE frame and ties. row_number/rank/dense_rank for top-N and dedup (filter outside), lag/lead for row-to-row, `value − row_number()` for islands.

9. DDL, DML and Constraints

Data types, `CREATE`, keys and constraints, identity and UUIDs, safe `ALTER`, `DROP`/`TRUNCATE`/`DELETE`, `INSERT`/`UPDATE`/`DELETE`, upsert and `MERGE`, backups.

Must-know

- **Types validate as well as store** — `numeric` for money, `integer`/`bigint` for ids, `text`, `timestamptz`, `date`, `boolean`. Not `float` for money, not `text` for dates, not `varchar(255)` by habit.
- **Keys** — **super key** (any unique set) ⊇ **candidate key** (minimal) → one **primary key** (unique + NOT NULL, one per table) + **alternate keys** (enforced with `UNIQUE`). **Composite** = several columns. **Natural** (email, ISBN) vs **surrogate** (identity/UUID).
- **Constraints** — `NOT NULL`, `DEFAULT`, `UNIQUE` (NULLs don't clash — several allowed; SQL Server allows one), `PRIMARY KEY`, `CHECK` (rejects only FALSE; NULL passes), `FOREIGN KEY`, PG `EXCLUDE` (e.g. overlapping bookings).
- **FK actions** — `NO ACTION` (default, checked at end of statement, deferrable), `RESTRICT` (immediate), `CASCADE`, `SET NULL`, `SET DEFAULT`.
- **PostgreSQL does not index FK columns automatically** → each parent delete scans the child table. Create one.
- **Identity** — `GENERATED ALWAYS AS IDENTITY` (preferred over legacy `serial`); `BY DEFAULT` lets you supply values. Sequences have **gaps** (rollbacks don't return numbers). `gen_random_uuid()` (PG 13+) for distributed ids. `GENERATED ALWAYS AS (expr) STORED` = computed column (PG 12+).
- **Safe ALTER on live tables** — add nullable column → backfill → `SET NOT NULL`; `ADD CONSTRAINT … NOT VALID` then `VALIDATE CONSTRAINT`; `CREATE INDEX CONCURRENTLY`; type changes rewrite the table under an exclusive lock.
- **DDL is transactional in PG** (rollback-able); MySQL/Oracle commit implicitly.
- **`UPDATE … SET`** expressions all see the **old** row values. PG: `UPDATE t … FROM s`, `DELETE FROM t USING s`; all DML supports `RETURNING`.
- **Soft delete** — `deleted_at`; every query must filter (view/RLS); use a **partial unique index** `WHERE deleted_at IS NULL`.

- **Upsert** — `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col` / `DO NOTHING`: atomic and concurrency-safe via the unique index. `MERGE` (PG 15+, SQL Server, Oracle) — multi-branch, can delete, not safe against concurrent inserts on its own.
- **Clustered index** — rows stored in index order (SQL Server PK default; InnoDB always the PK). PG has none: heaps; `CLUSTER` re-sorts once.

Key facts & numbers

	DELETE	TRUNCATE	DROP
Kind	DML	DDL	DDL
Removes	chosen rows	all rows	rows + table
WHERE	yes	no	no
Speed	slow, row by row, logged	fast (releases files)	fast
Row triggers	fire	don't (PG has ON TRUNCATE)	n/a
Identity	continues	PG continues unless `RESTART IDENTITY`; MySQL/SQL Server reset	gone
Rollback	yes	PG & SQL Server yes; **MySQL & Oracle no**	same as TRUNCATE
Lock	row locks	exclusive table	exclusive table

Dialect	Auto-increment	Get generated id	Upsert
PostgreSQL	`IDENTITY`, `serial`	`RETURNING`	`ON CONFLICT … DO UPDATE / NOTHING`, `MERGE` 15+
MySQL	`AUTO_INCREMENT`	`LAST_INSERT_ID()`	`ON DUPLICATE KEY UPDATE`, `INSERT IGNORE`
SQL Server	`IDENTITY(1,1)`	`OUTPUT inserted.*`, `SCOPE_IDENTITY()`	`MERGE`
SQLite	`INTEGER PRIMARY KEY`	`RETURNING` (3.35+)	`ON CONFLICT DO UPDATE` (3.24+), `INSERT OR REPLACE`

Enforcement	PG	MySQL	SQL Server	SQLite
CHECK	yes	8.0.16+	yes	yes
FK	yes	yes	yes	only with `PRAGMA foreign_keys = ON`
FK index auto	**no**	yes	no	no

Backup	What	Tools	Good for
Logical	schema + data as SQL/archive	`pg_dump` (`-F p/c/d/t`), `pg_restore` (`-j` parallel)	small/medium, single tables, cross-version
Physical	data files	`pg_basebackup`, snapshots, pgBackRest	large DBs, fast restore, same major version
PITR	base backup + archived WAL	WAL archiving	restore to any moment

Only `-F d` (directory) dumps in parallel. Rules: choose by **RPO/RTO**, 3-2-1 copies, encrypt, **test restores**; replication does not protect against a bad `DELETE`.

Code to remember

```
CREATE TABLE members (
  member_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email      text NOT NULL UNIQUE,
  club_id    int  REFERENCES clubs(club_id) ON DELETE SET NULL,
  fee        numeric(8,2) NOT NULL DEFAULT 0 CHECK (fee >= 0)
);
CREATE INDEX ON members (club_id);                -- FK index

INSERT INTO stock (sku, qty) VALUES ('A1', 5)
ON CONFLICT (sku) DO UPDATE SET qty = stock.qty + EXCLUDED.qty
RETURNING *;
```

In the interview

Q: Primary key vs unique key? **A:** PK: one per table, never NULL, the row's identity (often clustered in SQL Server/MySQL). UNIQUE: many per table, NULLs allowed.

Q: Natural or surrogate PK? **A:** Usually surrogate (bigint identity or UUID) for stability, plus UNIQUE on the natural key to keep the business rule. A changeable natural PK forces every FK to change.

Q: DELETE vs TRUNCATE vs DROP? **A:** DELETE: DML, chosen rows, triggers fire, table kept. TRUNCATE: DDL, all rows by deallocating storage, fast, no WHERE, rollback-able in PG/SQL Server not MySQL/Oracle. DROP removes the table.

Q: How do you upsert in PostgreSQL? **A:** `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col` — atomic via the unique index; MERGE can also delete but needs retry/locking for concurrent inserts.

Q: Add a NOT NULL column to a 100M-row table safely? **A:** Add nullable (or with a constant default), backfill in batches, then `SET NOT NULL` (or a `NOT VALID` CHECK then `VALIDATE`).

Q: `DELETE` without WHERE at 14:03 — recover? **A:** PITR: restore the base backup to a new server, replay WAL to 14:02:59, copy the lost rows back.

Q: Does pg_dump block a busy database? **A:** No, it reads a consistent snapshot; its light locks only block DDL on dumped tables.

Gotchas

- `ON DELETE CASCADE` everywhere → one delete wipes large parts of the database.
- Forgetting `NOT NULL` alongside `UNIQUE`/`CHECK` when a value is required.
- `CREATE TABLE AS SELECT` copies data, not constraints; `LIKE … INCLUDING ALL` copies structure.
- `TRUNCATE` isn't MVCC-safe: an older snapshot reading afterwards sees the table empty.
- `DELETE` to empty a huge table → use `TRUNCATE`; `CASCADE` without checking → test in `BEGIN … ROLLBACK`.
- Multi-row rules ("max 5 members per club") need triggers, `EXCLUDE` or transactions — constraints check single rows.
- Random UUID as a clustered key → page splits on insert.

TL;DR

Right types + declared keys and constraints (index FK columns in PG). Identity over serial; ALTER big tables in safe steps. DELETE vs TRUNCATE vs DROP; upsert with ON CONFLICT; backups: logical, physical, PITR — and test restores.

10. Data Modeling and ER Diagrams

Conceptual, logical and physical models; the ER model and its notations; attributes, cardinality and participation; weak entities; keys and integrity; ER → tables.

Must-know

- **Data model** — entities (nouns) → tables, attributes (facts) → columns, relationships (verbs) → foreign keys or junction tables. A fact you don't model is a question you can't answer.
- **Three levels** — **conceptual** (things + links, for the business), **logical** (attributes, keys, general types, normalized), **physical** (exact types, indexes, one DBMS). Only physical depends on the product.
- **Process** — requirements + the questions the system must answer → entities → relationships → conceptual review → logical → physical → validate with real data; loop back.
- **Attribute kinds** — **key** (underlined) → PK; **simple** → column; **composite** → one column per part; **multivalued** (double ellipse) → separate table; **derived** (dashed) → computed, not stored.
- **Degree** — number of entity types in a relationship: unary (self-FK), binary, ternary (table with three FKs).
- **Cardinality** — max per side: 1:1, 1:N, M:N. **Participation** — min: **total** (mandatory → FK `NOT NULL`) or **partial** (optional → nullable). `(min,max)` = (participation, cardinality).
- **Mapping** — 1:N → FK on the **many** side; M:N → **junction table** with both FKs as the PK; 1:1 → FK + `UNIQUE` (prefer the total-participation side).
- **Relationship attribute** — a fact needing both ends (`weekly_hours`, `grade`) → junction table (M:N) or many side (1:N).
- **Weak entity** — identified only with its owner: PK = owner key + **partial key**; owner FK `NOT NULL`, `ON DELETE CASCADE`. Chen: double rectangle, double diamond, dashed underline.
- **Keys** — superkey ⊇ candidate (minimal) ⊇ primary (chosen, NOT NULL); others are **alternate** keys with `UNIQUE`. **Prime attribute** = in any candidate key.
- **Integrity** — domain (type + CHECK), **entity** (no NULL in PK), key (uniqueness), **referential** (FK matches a row or is NULL — no orphans).
- **Chen vs crow's foot** — Chen (rectangles, ellipses, diamonds; teaching) vs crow's foot (attributes in boxes, cardinality in line ends; industry and mermaid).

Key facts & numbers

Crow's foot (mermaid)	Meaning
`|o` / `o|`	zero or one (0,1)
`||`	exactly one (1,1)
`}o` / `o{`	zero or more (0,N)
`}|` / `|{`	one or more (1,N)

ER construct	Relational result
Strong entity	table, key → PK
Composite attribute	one column per part (not the whole too)
Derived attribute	no column (query/view); store only if expensive or frozen in time (order total)
Multivalued attribute	own table: owner key + value, both in PK, CASCADE
1:N	FK on many side
M:N	junction table (business name: `enrolment`, `booking`)
1:1	FK + UNIQUE — or merge into one table
Unary / ternary	self-FK / table with three FKs
Weak entity	owner key + partial key, owner FK NOT NULL + CASCADE
Total / partial participation	FK NOT NULL / nullable

Best practice	Rule
Questions first	a model is correct only relative to the questions it answers
Rules in the database	every rule → PK, FK, NOT NULL, UNIQUE, CHECK
Types on purpose	dates as date, money as numeric
Keys	surrogate PK + UNIQUE on the natural key; keep keys narrow
Normalize	3NF by default; denormalize only for a measured slow query
Index	only for the real queries
Naming	one style; FKs named after target; role names (`origin_airport_id`)

In the interview

Q: Walk me through designing a schema. **A:** Requirements and the questions it must answer; nouns → entities, verbs → relationships; conceptual review with the business; then attributes, keys and normalization; then types and indexes for the named queries; validate with real data.

Q: Entity or attribute? **A:** Entity if it carries several facts and has many instances; a single value (a date, a colour) is an attribute.

Q: Where does the FK go in 1:N and why? **A:** On the many side; on the one side it would need a list in a cell (violates 1NF).

Q: How do you enforce 1:1? **A:** FK + UNIQUE on it (or share the PK). Would you merge? Yes, unless access control, row width or sparse optional data justify two tables.

Q: Can a plain FK enforce "every department has at least one employee"? **A:** No, an FK constrains the child side; needs a deferred constraint trigger or app logic.

Q: Is `{id, name}` a candidate key? **A:** It's a superkey, not a candidate: `name` can be dropped.

Q: UUID vs bigint surrogate? **A:** UUIDs can be generated anywhere without coordination but are 16 vs 8 bytes and random ones scatter B-tree inserts; time-ordered UUIDv7 reduces that.

Q: Where does `grade` go in student–course? **A:** In the enrolment junction table.

Gotchas

- A list in one column or `phone1, phone2, phone3` → separate table.

- One 60-column mostly-empty table = several unseparated entities; over-normalizing to seven joins per everyday question is a real cost too.
- M:N converted with an FK on one side → only right if it was really 1:N.
- Keeping `full_name` *and* first/last parts → copies disagree.
- Surrogate key on a junction table without `UNIQUE` on the pair → double enrolments.
- Weak entity with a surrogate key but no `UNIQUE(owner, partial)` → the rule is enforced nowhere.
- Showing column types to business reviewers; calling a diagram with keys "conceptual".
- Creating child tables before parents in migrations.

TL;DR

Model before tables: entities, attributes, relationships at conceptual → logical → physical levels. Degree, cardinality and participation are separate questions. Map 1:N → FK on many, M:N → junction, 1:1 → FK + UNIQUE, weak → owner key + partial key, multivalued → table; then normalize.

11. Normalization

Functional dependencies, Armstrong's rules, closures and keys, minimal cover, anomalies, 1NF → 5NF, lossless decomposition, and denormalization.

Must-know

- **Functional dependency** `X -> Y` — rows equal on X are equal on Y. A business rule: data can disprove it (one counter-pair), never prove it. X is a **superkey** iff X determines every column.
- **FD types** — trivial (RHS ⊆ LHS), **full** (needs the whole composite key), **partial** (part of a key → non-prime; kills 2NF), **transitive** (key → non-key → non-prime; kills 3NF), non-superkey determinant (kills BCNF), **multivalued** `X ->> Y` (kills 4NF).
- **Armstrong's axioms** — reflexivity, augmentation, transitivity: **sound and complete**. Derived: union, decomposition, pseudotransitivity.
- **Attribute closure X^+** — start with X, fire every FD whose whole LHS is present, loop until stable. Superkey ⟺ X^+ = all; candidate key = minimal superkey; `X -> Y` implied ⟺ Y ⊆ X^+. Attributes never on a right side are in **every** key.
- **Minimal cover** — single-attribute RHS, no extraneous LHS attributes, no redundant FDs; input to 3NF synthesis.
- **Anomalies** — **insertion** (can't add a product nobody ordered), **update** (customer moves → many rows), **deletion** (delete only order → lose the customer). Cause: a fact stored more than once.
- **Prime attribute** — in some candidate key.
- **Lossless join** — shared columns must be a key of one side, otherwise the join invents **spurious tuples**. Verify with `EXCEPT` both ways. **Dependency preservation** — each FD checkable within one table.
- **3NF vs BCNF** — 3NF allows X → A when A is prime; BCNF doesn't. They differ only with **overlapping composite candidate keys**. BCNF split is lossless but may lose an FD — why 3NF is the default stop.
- **Denormalization** — controlled redundancy for reads, only after normalizing and measuring a bottleneck indexes didn't fix; every copy needs an enforced maintenance path. Snapshots (price paid) are facts, not redundancy.

Key facts & numbers

Form	Rule	Removes
1NF	every cell atomic (no lists, no repeating groups)	lists in cells
2NF	no non-prime attribute depends on **part** of a candidate key	partial deps (only possible with composite keys)
3NF	for every non-trivial X → A: X superkey **or** A prime	transitive deps (any key shape)
BCNF	every non-trivial FD's determinant is a superkey	remaining FD redundancy
4NF	every non-trivial MVD's determinant is a superkey	independent lists (cross product: dishes × zones)
5NF	no join dependency not implied by keys	facts needing three tables (supplier-part-project)

Algorithm	Lossless	Dependency-preserving
3NF synthesis (Bernstein): minimal cover → one table per LHS → add a key table if none → drop contained	always	**always**
BCNF decomposition: split on violating X → Y into X^+ and (R − X^+) ∪ X	always	not always

Denormalization technique	Keeping it correct
Counter/summary column (`like_count`)	trigger in same transaction or batch recompute
Materialized view	`REFRESH` (stale between)
Duplicated column to avoid a join	trigger/app code on every change — riskiest
Star schema (warehouse)	rebuilt by ETL
External cache (Redis)	expiry / invalidation

Worked example: R(A,B,C,D,E), F = {A→B, BC→D, D→A, C→E}. C is never on a right side → in every key. Keys **AC, BC, CD**; E non-prime; C → E is partial → **only 1NF**. 3NF synthesis: {A,B}, {B,C,D}, {A,D}, {C,E}.

In the interview

Q: Explain 1NF, 2NF, 3NF. **A:** Atomic values; no non-key attribute depends on part of a composite key; no non-key attribute depends on another non-key attribute (formally: every X → A has X a superkey or A prime). 3NF by default: removes update anomalies and always has a lossless, dependency-preserving decomposition.

Q: 3NF violation with a single-column key? **A:** `employee(emp_id, dept_id, dept_name)` — `dept_id -> dept_name`.

Q: Example of 3NF but not BCNF? **A:** (patient, specialty, doctor) with doctor → specialty; splitting loses {patient, specialty} → doctor.

Q: How do you test if X is a key? **A:** Compute X^+; if it's everything X is a superkey; if no smaller subset is, it's a candidate key.

Q: Is a table with only key columns in 3NF? **A:** Yes (no non-prime attributes); check BCNF separately.

Q: When do you denormalize? **A:** For a measured read bottleneck indexing didn't fix: trigger-maintained counters, materialized views, star schemas — with one enforced maintenance path.

Q: Is storing an order total denormalization? **A:** If recomputable from current data, yes; if it freezes historical prices, it's its own fact.

Gotchas

- Comma-separated ids in a column ("**Jaywalking**", *SQL Antipatterns*) → no joins, no FKs; numbered columns `item1…item5` too.
- 1NF doesn't mean "no duplicate values".
- Checking 2NF only against the primary key → must check **every** candidate key and non-prime attributes.
- Splitting off only one part's dependents → still not 2NF.
- A lossy split (shared columns not a key) → spurious rows.
- Denormalizing write-heavy tables or "because joins are slow" → joins on indexed keys are what RDBMSs do well.

TL;DR

FDs → closures → keys → check 1NF/2NF/3NF (BCNF/4NF/5NF when their signatures appear) → lossless, dependency-preserving splits. Stop at 3NF by default; denormalize only with measurements and a maintenance path.

12. Indexes, Storage and Query Performance

Pages and heaps, compression, B+ trees, hash indexes, Bloom filters, GIN/GiST/BRIN, reading `EXPLAIN`, composite/covering/partial indexes, anti-patterns, keyset paging, join algorithms, statistics.

Must-know

- **Performance = pages touched.** A table is a file of fixed-size slotted pages (PG 8 KB); 1M orders = 8,662 pages; a Seq Scan reads them all.
- **Row store vs column store** — row: whole rows together (OLTP; PG, InnoDB); column: one column's values together (analytics; ClickHouse, Redshift, Snowflake, Parquet), compresses well with RLE, dictionary, delta encoding, bit-packing.
- **File organization** — heap (unordered, PG), sorted, hashed, **clustered/index-organized** (InnoDB, SQL Server clustered, Oracle IOT). One physical order per table; **correlation** decides whether an index read touches ~100 or ~8,400 pages.
- **Index kinds** — primary (on sort key, sparse), clustering, secondary (dense); **dense** = entry per record, **sparse** = entry per block (only on the sort key). **Multilevel** indexes → **B+ tree**.
- **B+ tree** — balanced, nodes are pages with hundreds of keys → 3–4 levels for a billion rows; values in **linked leaves** so ranges walk the leaf chain; inserts split nodes and keep leaves at equal depth. PG 13+ deduplicates repeated keys.
- **Hash index** — equality only; no ranges, ordering, or unique constraints in PG; crash-safe since PG 10.
- **Bloom filter** — bit array + k hashes; "no" is certain, "yes" is probable; **~10 bits/key ≈ 1 % false positives**; too many hashes hurt. Skips disk reads in LSM stores and joins.
- **Scans** — **Seq** (many rows), **Index** (few rows, ordered), **Index Only** (all needed columns in the index + visibility map says all-visible), **Bitmap** (many scattered rows; combinable with BitmapAnd/Or).
- **Composite index** — **leftmost prefix** rule; **equality columns first, range column last**; `(a,b)` helps `ORDER BY b` only when `a` is fixed by equality.
- **Covering** `INCLUDE (cols)` → index-only scans; **partial** `WHERE status = 'pending'` for hot subsets; **expression** `(lower(email))` for function predicates.

- **Sargable** predicates — no functions or arithmetic on the indexed column; half-open ranges instead of `created_at::date = …`; leading `%` needs a trigram GIN index.
- **N+1 queries** — one query per row from app loops; replace with one join or `IN` / `ANY`.
- **Keyset pagination** — OFFSET cost grows linearly with page number; keyset `(created_at, id) < (:ts, :id)` on a `(created_at, id)` index stays ~4 pages (79 ms → 0.02 ms). Unique tie-breaker required; can't jump to page N.
- **Cost-based optimizer** — estimates rows from statistics; compare estimated vs actual rows in `EXPLAIN ANALYZE`; big gaps → `ANALYZE`, higher statistics target, extended statistics.

Key facts & numbers

Index type (PG)	Answers	Use
B-tree (default)	`= < > BETWEEN`, prefix LIKE, ORDER BY	almost everything
Hash	`=`	long equality-only keys
GIN (inverted)	`@>`, `?`, full-text `@@`, `%x%` with pg_trgm	tags, JSONB, search
GiST	overlap `&&`, containment, nearest `<->`	PostGIS, ranges, exclusion constraints
SP-GiST	partitioned space	points, IP ranges, prefixes
BRIN	min/max per 128-page block range	huge append-only time columns

EXPLAIN field	Meaning
`cost=a..b`	startup..total estimate (seq page 1.0, random `random_page_cost` 4.0)
`rows` vs `actual rows`	estimate vs real — big gap = bad stats
`loops`	times a node ran; per-loop numbers
`Buffers: shared hit/read`	pages from buffer pool / from OS
`Rows Removed by Filter`	wasted work; `Index Cond` = useful work

Join	Best when
Nested loop	small outer, indexed inner
Hash	large unsorted equality joins (needs `work_mem`)
Merge	inputs already sorted / output needs order

Anti-pattern	Fix
`created_at::date = '2024-06-01'`	`>= '2024-06-01' AND < '2024-06-02'`
`id + 1 = 500001`	`id = 500000`
`LIKE '%x%'`	trigram GIN
`lower(email) = …`	expression index
`a = … OR b = …` (b unindexed)	index both (BitmapOr) or UNION

Planner stats (pg_stats)	Used for
null_frac, n_distinct (negative = fraction; −1 unique)	IS NULL, =, GROUP BY size
most_common_vals/freqs	skewed values
histogram_bounds	ranges
correlation	index-scan cost

	PG	InnoDB	SQL Server	SQLite
Storage	heap + indexes	clustered on PK	heap or clustered	B-tree by rowid
Page	8 KB	16 KB	8 KB	4 KB
Partial index	yes	no	filtered index	yes
Plan	EXPLAIN (ANALYZE, BUFFERS)	EXPLAIN ANALYZE 8.0.18+	actual plan	EXPLAIN QUERY PLAN

Code to remember

```
CREATE INDEX CONCURRENTLY orders_cust_created_idx
  ON orders (customer_id, created_at DESC) INCLUDE (amount);
CREATE INDEX ON orders (created_at) WHERE status = 'pending';   -- partial
CREATE INDEX ON customers (lower(email));                       -- expression

EXPLAIN (ANALYZE, BUFFERS)
SELECT amount FROM orders WHERE customer_id = 777
ORDER BY created_at DESC LIMIT 3;
```

In the interview

Q: How does a B+ tree index work? **A:** Balanced tree of page-sized nodes with hundreds of keys, so 3–4 levels even for a billion rows; values in linked leaves, so range scans walk the leaves; inserts split nodes upward.

Q: Why not index everything? **A:** Every index is maintained on writes, costs space and cache; low-selectivity indexes get ignored anyway. Find unused ones via `pg_stat_user_indexes.idx_scan = 0`.

Q: Clustered vs non-clustered? **A:** Clustered stores rows in key order (one per table); non-clustered is a separate structure with pointers. PG indexes are all non-clustered.

Q: Why would the planner ignore my index? **A:** Low selectivity (95 % rows match), poor correlation, tiny table, stale stats, a function or type mismatch on the column, or the query needs most rows.

Q: Order columns in a composite index? **A:** Equality columns first, range last; leftmost prefix must be used. Test: `(customer_id, created_at)` can't serve a `created_at`-only range.

Q: B-tree vs LSM? **A:** B+ trees update pages in place (read-optimized); LSM appends and merges (write-optimized, reads need Bloom filters).

Q: How do you tune a slow query? **A:** `EXPLAIN (ANALYZE, BUFFERS)`, compare estimated vs actual rows, find the node with most pages/time, fix stats or add/reshape an index, make predicates sargable, re-measure.

Gotchas

- Separate `(a)`, `(a,b)`, `(a,b,c)` indexes → the last covers the others.
- PG doesn't auto-index FK columns → joins and parent deletes scan the child.

- Random UUID keys → random leaf inserts, page splits; in InnoDB/SQL Server the wide clustered key is copied into every secondary index.
- First (cold) run timings mislead; compare buffer counts and plan shapes.
- Compression costs CPU; PG compresses (TOAST) only rows over ~2 KB.
- Keyset with a non-unique sort key skips/repeats rows.

TL;DR

Count pages. B+ trees for most things, GIN/GiST/BRIN for special cases, Bloom filters to skip reads. Read EXPLAIN (ANALYZE, BUFFERS); composite = equality first, range last; INCLUDE/partial/expression indexes; sargable predicates; keyset paging; fix bad estimates with ANALYZE.

13. Transactions and Concurrency

ACID, isolation anomalies and levels, lost updates, write skew and SSI, locks and 2PL, deadlocks, `SKIP LOCKED` queues, timestamp ordering, MVCC.

Must-know

- **Transaction** — all-or-nothing unit: `BEGIN … COMMIT/ROLLBACK`; `SAVEPOINT` / `ROLLBACK TO SAVEPOINT` for partial undo; without `BEGIN` each statement autocommits (and a multi-row UPDATE is still atomic).
- **ACID** — **A**tomicity (undo/WAL; shadow copy explains the idea), **C**onsistency (constraints + correct code — DDIA calls it an application property), **I**solation (locks, MVCC, SSI), **D**urability (WAL fsynced before commit is acknowledged, checkpoints, replication).
- **ACID "C" ≠ CAP "C"** (CAP means linearizability across replicas). ACID "atomic" is about failures, not concurrency.
- **Anomalies** — **dirty read** (see uncommitted), **dirty write**, **non-repeatable read** (same row changes), **read skew** (inconsistent multi-row read: 500 + 400 = 900 when truth was 800), **phantom** (the *set* of matching rows changes), **lost update** (two read-modify-writes, one overwritten), **write skew** (both read, decide, write *different* rows; invariant breaks — on-call doctors).
- **PostgreSQL levels** — Read Uncommitted = Read Committed (no dirty reads ever); **Read Committed** (default): new snapshot per statement; **Repeatable Read** = snapshot isolation, one snapshot from the **first statement**, no phantoms, aborts lost updates (40001), allows write skew; **Serializable** = SI + **SSI** tracking, aborts one of a write skew.
- **Lost-update fixes** — atomic `UPDATE … SET x = x − 1`; `SELECT … FOR UPDATE`; Repeatable Read + retry; **version column** (CAS: `UPDATE … WHERE version = 7`, 0 rows → retry) for long think time.
- **Write-skew fixes** — Serializable + retry, `FOR UPDATE` on the rows the decision depends on, or a unique/**exclusion constraint**. `FOR UPDATE` can't lock a row that doesn't exist yet (phantom) → constraint, Serializable, or materialized slot rows.
- **Retry** SQLSTATE **40001** (serialization failure) and **40P01** (deadlock) by rerunning the whole transaction.
- **Locks** — **S** (shared, read) compatible only with S and IS; **X** (exclusive) with nothing; **intention locks** IS/IX on tables let the manager check table-vs-row conflicts without scanning rows.
- **2PL** — growing phase then shrinking → serializable. **Strict 2PL** holds X locks to commit (no cascading aborts); **rigorous** holds all; **conservative** takes all up front (no deadlocks).
- **Phantom prevention** — predicate locks (theory; SSI's SIREAD is a non-blocking cousin) or **index-range / next-key locks** (InnoDB, SQL Server).
- **Deadlock** — cycle in the wait-for graph; detected (PG after `deadlock_timeout` = 1 s) and a victim aborted. Prevent: lock in a consistent order (sort by PK), keep transactions short. Wait-die / wound-wait use timestamps.

- **`FOR UPDATE SKIP LOCKED`** — safe multi-worker job queue; `NOWAIT` fails immediately (55P03). Never for reports.
- **Timestamp ordering** — read rejected if a younger txn wrote the item (TS < WTS); write rejected if a younger one read or wrote it; loser restarts. No deadlocks, many aborts, starvation; Thomas write rule skips obsolete writes.
- **MVCC** — multiple row versions; readers use their snapshot, **readers and writers don't block each other** (writers still block writers). PG: update inserts a new tuple, sets `xmax` on the old (`xmin` = creator); **VACUUM** removes dead tuples and freezes XIDs (32-bit, wraparound ~2 billion). InnoDB/Oracle: update in place + **undo log**.

Key facts & numbers

Standard level	Dirty read	Non-repeatable	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	—	possible	possible
Repeatable Read	—	—	possible
Serializable	—	—	—

PostgreSQL	Non-repeatable	Phantom	Lost update	Write skew
Read Committed	yes	yes	yes (app read-modify-write)	yes
Repeatable Read	no	**no**	no (aborts)	**yes**
Serializable	no	no	no	no (aborts)

Engine	Default	Note
PostgreSQL	Read Committed	RR = SI, Serializable = SSI
MySQL InnoDB	**Repeatable Read**	locking reads see latest + next-key locks; does not abort lost updates
SQL Server	Read Committed (locking)	optional RCSI / SNAPSHOT row versions
Oracle	Read Committed	its "SERIALIZABLE" is snapshot isolation
SQLite	Serializable	one writer per database file

Need	Use
Normal OLTP, single-statement updates	Read Committed
Consistent multi-query report	`REPEATABLE READ READ ONLY`
Read-then-write same row	`FOR UPDATE` or RR + retry
Cross-row invariant	Serializable + retry, or explicit locks/constraints

	Optimistic (OCC)	Pessimistic (PCC)
Assumes	rare conflicts	common conflicts
Conflict cost	abort + redo	waiting
Deadlocks	none	possible
Fits	read-heavy, long think time, distributed (Git)	hot rows: stock, seats, banking

PG table lock	Taken by	Blocks
ACCESS SHARE	SELECT	only ACCESS EXCLUSIVE
ROW EXCLUSIVE	INSERT/UPDATE/DELETE	SHARE and stronger
SHARE	CREATE INDEX	writers
ACCESS EXCLUSIVE	most ALTER, DROP, TRUNCATE, VACUUM FULL	everything, even SELECT

After an error mid-transaction: PG aborts the whole transaction until ROLLBACK; MySQL/SQLite undo only the failing statement.

Code to remember

```sql
-- job queue worker
BEGIN;
SELECT id, payload FROM jobs
WHERE status = 'ready'
ORDER BY id
FOR UPDATE SKIP LOCKED
LIMIT 1;
UPDATE jobs SET status = 'running' WHERE id = :id;
COMMIT;

-- optimistic lock
UPDATE products SET price = :p, version = version + 1
WHERE id = :id AND version = :v;      -- 0 rows → someone else won → retry
```

In the interview

Q: What is a transaction / ACID? **A:** A unit of work that fully commits or fully rolls back; atomicity, consistency, isolation, durability, backed by the log, constraints, locks/MVCC and fsync.

Q: Non-repeatable read vs phantom? **A:** Same row changed value vs the set of matching rows changed (insert/delete).

Q: How do you prevent lost updates? **A:** Atomic SQL arithmetic; `FOR UPDATE`; Repeatable Read where PG aborts the second writer; or a version column. "Like" counter → atomic update; user editing for minutes → version column (never hold locks across think time).

Q: What is write skew? **A:** Two transactions read the same data, decide, and write different rows so no write conflict but an invariant breaks; snapshot isolation allows it; Serializable (SSI) aborts one.

Q: Why not always Serializable? **A:** More aborts (or blocking with 2PL) and apps must retry; most workloads are fine at Read Committed with targeted locks.

Q: Why write the WAL and not the data pages at commit? **A:** One sequential log append is far cheaper than scattered page writes, and the log alone can redo the change.

Q: How are deadlocks handled? **A:** Detected via the wait-for graph; a victim is aborted (40P01) and retried. Prevent by consistent lock order and short transactions.

Q: Explain MVCC. **A:** Rows carry versions tagged by transaction; readers see their snapshot, so readers never block writers. PG sets xmax on old versions and VACUUM cleans up; costs: bloat, and long transactions hold back cleanup.

Q: Build a job queue in SQL? **A:** `SELECT … FOR UPDATE SKIP LOCKED LIMIT 1`, mark it running, commit; good when jobs are created in the same transaction as business data; otherwise use a broker.

Gotchas

- Two halves of a transfer as two autocommitted statements.
- Catching an error and still COMMITting → PG silently rolls back; MySQL may commit the half that worked.
- Rollback doesn't unsend an email or uncharge a card → outbox table, send after commit.
- Turning off `fsync` → crash can corrupt the database. Async replicas don't make a commit durable.
- Long "idle in transaction" sessions → bloat; set `idle_in_transaction_session_timeout`. Never disable autovacuum (XID wraparound).
- ALTER on a busy table queues every later query behind ACCESS EXCLUSIVE → `SET lock_timeout = '2s'`.
- Multi-query report at Read Committed → totals that never existed.
- Locking rows in unsorted input order → deadlocks.

TL;DR

ACID via log, constraints, isolation, fsync. Know the anomalies and which level stops which (PG RR = snapshot isolation, allows only write skew; Serializable = SSI). Fix lost updates with atomic SQL/FOR UPDATE/versions; retry 40001/40P01. 2PL locks vs MVCC versions; SKIP LOCKED for queues.

14. Advanced SQL and Using SQL from Code

Functions, procedures, triggers, JSONB, arrays, full-text search, pivots, roles and row-level security, SQL injection, drivers, pooling and ORMs.

Must-know

- **UDF** — `LANGUAGE sql` for simple expressions (can be inlined), `plpgsql` for branches/loops. **Volatility**: `IMMUTABLE` (same inputs → same result forever; required to index an expression), `STABLE` (constant within a statement, may read tables), `VOLATILE` (default).
- **Procedure** — `CALL p()`, returns via `INOUT`, can `COMMIT` inside (PG 11+, only when not called inside a transaction block) → batch jobs. Functions always run in the caller's transaction.
- **Trigger** — automatic function on INSERT/UPDATE/DELETE: `BEFORE` (can modify or reject; must `RETURN NEW`), `AFTER` (reacts, sees final row), `INSTEAD OF` (views); `FOR EACH ROW` or `FOR EACH STATEMENT` (+ transition tables for bulk audit). Runs in the same transaction → rolls back with it.
- **Use triggers for** audit logs, `updated_at`, cross-table integrity checks; **not** for business workflows or external calls (use an **outbox** table). Prefer a constraint when one works.
- `jsonb` — parsed binary, indexable (GIN), keys reordered, last duplicate wins; `json` keeps exact text. `->` returns jsonb, `->>` **returns text**, `#>>` path, `@>` contains, `?` key exists. Keep core/filtered/joined fields as real columns (schemaless blob = EAV antipattern).
- **Arrays** — 1-based; `= ANY(arr)`, `@>` contains all, `&&` overlaps, `unnest`, `array_agg`; GIN index. Need joins/FKs on elements → child table.
- **Full-text search** — `to_tsvector` (stemmed **lexemes**, stop words dropped) `@@` `tsquery`; store as weighted generated column + GIN; `websearch_to_tsquery` for user input; `ts_rank`, `ts_headline`. Elasticsearch only for fuzzy/facets/huge scale.
- **Pivot** — group by row key + one conditional aggregate per column (`FILTER` / `CASE`); `crosstab` (tablefunc); **unpivot** with `CROSS JOIN LATERAL (VALUES …)` or `UNION ALL`. Dynamic columns → generate SQL in code or return JSON.
- **Security** — least privilege: group roles hold grants, login roles are members, app gets only needed DML; `ALTER DEFAULT PRIVILEGES` for future tables.
- **Row-level security** — `CREATE POLICY … USING (read visibility) WITH CHECK (write validity)`; a role with SELECT but no policy sees **0 rows**; **superusers and table owners bypass RLS** (`FORCE ROW LEVEL SECURITY`). Multi-tenant: policy keyed on a session setting.

- **SQL injection** — input concatenated into SQL; fix with **parameterized queries** (values never change the parse tree); identifiers via allow-list + `%I` / `quote_ident`; least privilege limits blast radius.
- **Drivers** — psycopg `%s` (autocommit **off** by default), sqlite3 `?`, JDBC `?` via `PreparedStatement` (autocommit **on** by default). Money as `Decimal` / `BigDecimal`. JDBC `getInt` returns 0 for NULL → `wasNull()`.
- **Connection pooling** — reuse connections; more is not faster; start ≈ (cores × 2) + effective spindles (HikariCP), then measure pool wait time. PG `max_connections` default 100. Many replicas → external pooler (PgBouncer, RDS Proxy). Use `SET LOCAL` to avoid state leaks.
- **ORMs** — fast CRUD, parameterized by default, but hide SQL → **N+1** and bloated joins; raw SQL for complex/hot paths; fix N+1 with eager/batched loading or `= ANY($1)`.

Key facts & numbers

	Function	Procedure
Invoked by	`SELECT f()` / any expression	`CALL p()`
Returns	value or set	nothing (INOUT/OUT)
COMMIT inside	no	yes (outside a txn block)

Triggers	PG	MySQL	SQL Server	SQLite
Row-level	yes	only kind	no	only kind
Statement-level	yes + transition tables	no	only kind (`inserted` / `deleted`)	no
INSTEAD OF	views	no	tables & views	views

JSON	Type	Extract	Index
PostgreSQL	json, jsonb	`->>`, `#>>`, jsonpath	GIN; B-tree expression
MySQL 5.7+	JSON	`->>`, `JSON_EXTRACT`	generated columns, multi-valued (8.0.17+)
SQL Server	nvarchar (+ native json 2025)	`JSON_VALUE`, `OPENJSON`	computed column

RLS	PG	MySQL	SQL Server	Oracle
Mechanism	`CREATE POLICY` (9.5+)	none (views/app)	security policies	VPD

Defence in depth vs injection	
Query	bind every value; allow-list identifiers
Account	least-privilege role
Data	RLS
Errors	don't show raw DB errors

Code to remember

```
CREATE FUNCTION set_updated_at() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$;
CREATE TRIGGER trg_upd BEFORE UPDATE ON products
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE INDEX ON products USING gin (specs);           -- jsonb @>
SELECT name FROM products WHERE specs @> '{"brand":"Nimbus"}';

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_rows ON orders
  USING (tenant_id = current_setting('app.tenant')::int);
```

```
cur.execute("SELECT * FROM users WHERE email = %s", (email,))   # never f-strings
```

In the interview

Q: Function vs view? **A:** A view has no parameters; a set-returning function does. For a fixed query a view is simpler and fully optimisable.

Q: BEFORE vs AFTER trigger? **A:** BEFORE can modify or reject the row; AFTER sees the final row (after constraints) and reacts (audit). Bulk audit → statement-level trigger with transition tables.

Q: json vs jsonb? **A:** jsonb is parsed, indexable and fast to query; json keeps exact text. Use jsonb for variable attributes, promote important fields to columns.

Q: How do you prevent SQL injection? **A:** Parameterized queries for all values, allow-listed quoted identifiers for dynamic names, least privilege. ORMs are safe only if you avoid their raw-SQL string helpers. `LIKE ?` is injection-safe but `%`/`_` still act as wildcards.

Q: Design multi-tenant isolation. **A:** tenant_id on every table, RLS policy on a session setting, app role not owner, test with `SET ROLE`; USING = read, WITH CHECK = write.

Q: ORM or raw SQL? **A:** ORM for simple entity CRUD; SQL or a query builder for reports, windows, CTEs, JSONB and hot paths. Detect N+1 by counting queries per request.

Q: Why pool connections, and how big? **A:** Connections are expensive processes; a small pool (~cores × 2 + spindles) beats a big one; watch wait time and fix slow queries first; PgBouncer for many app replicas.

Q: Postgres FTS vs Elasticsearch? **A:** PG FTS: no extra system, transactionally fresh, stemming/ranking/phrases, fine to millions of docs. ES: typo tolerance, facets, synonyms at scale.

Gotchas

- Wrong volatility: table-reading function marked IMMUTABLE → stale results; pure function left VOLATILE → blocks optimisations.
- PL/pgSQL name clash `WHERE customer_id = customer_id` → prefix `p_`/`v_`.
- BEFORE row trigger returning NULL silently skips the row; triggers updating their own table can loop (`pg_trigger_depth()`).
- `specs -> 'brand' = 'Nimbus'` fails (jsonb vs text) → `->>` or `@>`; numbers from JSON compare as text unless cast.
- Index with `to_tsvector('english', …)` but query without the config → index unused. Raw input in `to_tsquery` → syntax errors.
- Testing RLS as superuser/owner proves nothing. PG ≤ 14 lets everyone CREATE in `public`.
- Leaking connections; holding a connection during an external API call.

TL;DR

Functions/procedures/triggers keep data rules near the data (small, honest volatility, no external work). JSONB, arrays and FTS with GIN handle semi-structured data and search. Least privilege + RLS + parameters secure it; pool connections and watch ORMs for N+1.

15. Scaling Databases

Scale up vs out, partitioning, sharding and consistent hashing, distributed queries, replication, consistency models, CAP/PACELC, consensus, WAL recovery and 2PC, HA and backups.

Must-know

- **Scale up first** (bigger machine, then read replicas and caching); **scale out** = **partitioning + replication** together. Network partition ≠ data partitioning.
- **Horizontal partitioning** splits rows; **vertical** splits columns (pieces share the PK 1:1; listing page read 128 kB instead of ~1.8 MB).
- **Partition pruning** — the planner skips partitions the `WHERE` excludes (check with `EXPLAIN`); plus cheap maintenance (drop a month instantly). Only helps queries filtering on the key. PG: unique/PK must include the partition key; indexes are **local** (no global index).
- **Sharding** — partitions on separate servers. **Shard key**: high cardinality/even load, matches the main query (one shard per query), co-locates related data, immutable. Without the key → **scatter-gather**; cross-shard transactions → 2PC.
- **Consistent hashing** — keys and shards on a ring, each key goes to the next shard clockwise; **virtual nodes** balance load. Going 4 → 5 shards: `hash % N` moves ~80 % of keys; 200 vnodes/shard moved ~17 % (ideal 20 %). Alternative: fixed logical partitions + directory (Citus, Vitess, Kafka).
- **Hot keys** — celebrity user: split key (`user123#0..9`) and merge on read, or cache. **Global ids**: UUIDv7, Snowflake (timestamp + machine + sequence), disjoint per-shard ranges.
- **Distributed queries** — push down predicates and partial aggregates (AVG as SUM + COUNT), co-located joins, **broadcast** small tables, **shuffle** by join key, **semi-join** ships keys not rows; per-shard top-N before a global `ORDER BY … LIMIT`.
- **Replication** — **sync**: no acknowledged commit lost, slower, blocks if the replica is down; **async**: fast, lag, may lose recent commits; semi-sync is common. Topologies: **single-leader** (default, no conflicts), **multi-leader** (multi-region writes, conflicts), **leaderless** (quorums **R + W > N**). Last-write-wins silently loses concurrent updates.
- **Consistency models** — **linearizable** (latest write, real-time order) > **sequential** (one global order) > **causal** (cause before effect; vector clocks) > **eventual** (converge when writes stop). **Session guarantees**: read-your-writes, monotonic reads, monotonic writes, writes-follow-reads.
- **Linearizable vs serializable** — single-object recency vs multi-object transactions as if one at a time; both = **strict serializability** (Spanner).
- **CAP** — during a partition, a replicated store gives up linearizable C or A; real choice CP vs AP. **PACELC** — else (no partition) trade **L**atency vs **C**onsistency.
- **Consensus** — Raft/Paxos: majority, **2f+1 nodes survive f crashes**; PBFT: **3f+1** for f Byzantine; Raft uses randomised election timeouts + majority votes. Use 3 or 5 nodes (4 tolerates only 1 failure). **Split brain** → majority quorums + **fencing**.
- **Recovery** — WAL flushed before commit ack; **checkpoints** bound replay; redo committed + undo uncommitted (PG: redo-only thanks to MVCC); LSNs order log records. `synchronous_commit = off` may lose the last fraction of a second, never corrupts.
- **2PC** — prepare (vote) then commit; **blocks** if the coordinator dies after prepare; 3PC isn't partition-safe; modern systems put the decision on a consensus group.

- **HA** — heartbeat → quorum confirms failure → promote most up-to-date standby → fence the old primary → redirect clients. State **RPO/RTO**.
- **Replicas are not backups** — bad writes replicate too.

Key facts & numbers

Partition strategy	Good for	Watch out
Range	time, archiving	hot newest partition
List	regions, tenants, residency	uneven sizes, new values
Hash	even spread (`user_id`)	no range pruning; changing N moves data
Composite	time + hash for high-volume series	many partitions
Round-robin	even bulk loads	key lookups scan all

Model	Guarantee	Use
Linearizable	latest write, real-time	balances, locks, leader election
Causal	cause before effect	comment threads
Eventual	converges	caches, counters, DNS

System	PACELC
Cassandra, Riak, DynamoDB (default)	PA / EL
Spanner, CockroachDB, etcd	PC / EC

Consensus	Tolerates	Cost
Paxos, Raft	$\lfloor (N-1)/2 \rfloor$ crashes	majority round trips
PBFT	$\lfloor (N-1)/3 \rfloor$ liars	$O(N^2)$ messages

Leader election	Messages	Note
Bully	up to $O(n^2)$	highest live id wins
Ring	~2n	slow on big rings
ZooKeeper / etcd / K8s Lease	—	ephemeral sequential znodes, lowest leads

Failover standby	Recovery	Cost
Cold	minutes–hours	cheapest
Warm	seconds–minutes	medium
Hot	seconds	highest

Backup	Restore needs
Full	that backup
Incremental (since last backup of any kind)	last full + every incremental in order
Differential (since last full)	last full + latest differential

Redundancy: RAID 1/10 (2×, mirrors), RAID 5/6 (1/2 parity disks), erasure coding 6+3 → 1.5× vs 3× replication.

Code to remember

```
CREATE TABLE events (id bigint, at timestamptz NOT NULL, payload jsonb)
  PARTITION BY RANGE (at);
CREATE TABLE events_2025_01 PARTITION OF events
  FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
EXPLAIN SELECT count(*) FROM events WHERE at >= '2025-01-10';  -- pruning
```

In the interview

Q: Partitioning vs sharding? **A:** Partitioning splits a table into pieces (can be one server; pruning, maintenance); sharding places partitions on different servers to scale writes and storage.

Q: How do you choose a shard key? **A:** High cardinality for even load, matches the main access path so most queries hit one shard, co-locates related rows, never changes. Avoid monotonic keys with range sharding and low-cardinality keys.

Q: Range vs hash for an events table? **A:** Range by time if queries are "recent" and old data is archived; hash by user_id for per-user queries and spread writes; time + hash composite for hot partitions.

Q: Why consistent hashing? **A:** `hash % N` remaps most keys when N changes; a ring with vnodes moves ~1/N and stays balanced.

Q: User doesn't see their own comment? **A:** A lagging async replica served the read → read-your-writes routing (read from leader after a write, or wait for the LSN).

Q: Explain CAP. **A:** Partitions happen, so choose CP (refuse some requests: banking, inventory) or AP (answer, reconcile: feeds, carts). PACELC adds latency vs consistency without partitions. A single Postgres isn't distributed, so CAP doesn't apply.

Q: Why 3 or 5 nodes? **A:** Majority of 4 is 3, so 4 tolerates one failure, same as 3.

Q: 2PC problems? **A:** Participants block holding locks if the coordinator fails after prepare; use consensus-backed commit or avoid cross-shard transactions (sagas).

Q: Accidental DROP TABLE at 14:02? **A:** Restore base backup to a separate instance, replay WAL to 14:01:59, copy the table back.

Gotchas

- Partition key the hot queries don't filter on → every partition scanned; partitioning tables under a few million rows (an index is simpler); forgetting next month's partition; thousands of tiny partitions.
- Sharding too early; forgetting global unique constraints, cross-shard joins and reports.
- Functions on remote columns block push-down (`EXPLAIN VERBOSE` → Remote SQL).
- Joining two large tables sharded on different keys in the hot path.
- Key-based incremental replication misses hard deletes (use tombstones).
- Leader election without fencing → split brain.

TL;DR

Scale up, then partition + replicate. Shard on a high-cardinality key matching the main query; consistent hashing with vnodes. Sync vs async replication, R + W > N; pick consistency per feature; CAP = CP or AP under partition. Raft/Paxos 2f+1; WAL + checkpoints for recovery; 2PC blocks; replicas aren't backups.

16. Beyond Relational

Why NoSQL, key-value, document, wide-column and graph stores, in-memory, NewSQL, time-series and spatial, big-data frameworks, and choosing a database.

Must-know

- **Why NoSQL** — flexible schemas, built-in scale-out, data stored the way it's read (no joins in the hot path). Many lean AP / **BASE**; the ACID–BASE divide is now a spectrum.
- **Key-value** — distributed hash map: O(1) get/put, easy to shard, no queries by value. Redis adds data structures (sorted sets) and TTLs. Caches, sessions, counters, rate limits, leaderboards; keep the durable truth elsewhere.
- **Document** — self-contained JSON documents in collections; model around queries. **Embed** what is owned, bounded and read together; **reference** what is shared or unbounded. Weak at many-to-many and cross-document constraints. MongoDB multi-document transactions since 4.0 (replica sets) / 4.2 (sharded).
- **Wide-column** (Cassandra, HBase) — **partition key** for placement + **clustering key** for order within the partition + LSM for fast writes; **one table per query**, no joins; tunable consistency (`ONE` / `QUORUM` / `ALL`). Bucket partitions by time to keep them bounded (~100 MB guidance).
- **Graph** — nodes and edges with properties; **index-free adjacency**: traversal cost ∝ edges touched, not database size. Cypher, Gremlin, SPARQL, ISO **GQL** (2024), SQL/PGQ (SQL:2023). Hard to shard, weak at bulk aggregates. Small cases → recursive CTEs.
- **In-memory** — RAM-first, microsecond access; durability via snapshots (Redis RDB) + append-only log (AOF) + replicas; cost per GB and size ceiling.
- **NewSQL / distributed SQL** — SQL + ACID + automatic sharding into ranges, each replicated by Raft/Paxos; cross-shard 2PC on consensus groups (no single blocking coordinator). Spanner: TrueTime + commit-wait, strict serializability; CockroachDB: Postgres-compatible, hybrid logical clocks, serializable default. Cost: write latency (majority, maybe cross-region).
- **Time-series** — partition by time, compress, retention, downsampling/continuous aggregates. PG: range partitions + BRIN + `date_bin`, or TimescaleDB.
- **Spatial** — points, lines, polygons; R-tree (GiST), quadtree, geohash/H3/S2; PostGIS.
- **Big data** — Hadoop: HDFS replicated blocks + MapReduce (map → shuffle → reduce, via disk); **combiner** = local reduce before shuffle (associative ops only). **Spark** keeps intermediates in memory (faster for iterative jobs). SQL on top: Hive (batch), Presto/Trino (interactive, federated), Spark SQL. Real time: **Kafka** (durable log) + **Flink** (stream processing).
- **Choosing** — start from access patterns and consistency needs; default to Postgres; add a specialised store only when it clearly earns its operational cost.

Key facts & numbers

Need	Consider	Postgres can go far with
Joins, constraints, transactions	PostgreSQL / MySQL	—
Cache, sessions, counters, rate limits	Redis (Memcached)	`UNLOGGED` tables
Flexible nested objects	MongoDB, Firestore, CouchDB	`jsonb` + GIN + CHECK
Massive time-ordered writes, multi-region	Cassandra / ScyllaDB, HBase	partitions + BRIN
Deep relationship queries	Neo4j, Neptune	recursive CTEs
Metrics, retention	TimescaleDB, InfluxDB, Prometheus	partitions + `date_bin` + BRIN
Maps, near me	PostGIS, Redis GEO	`point` / `box` + GiST
SQL + ACID beyond one node	CockroachDB, Spanner, YugabyteDB, TiDB	Citus
Analytics over billions of rows	BigQuery, Snowflake, Redshift, ClickHouse	replicas, summary tables
Search with facets	Elasticsearch / OpenSearch	`tsvector` + GIN

	Cassandra	HBase
Architecture	leaderless ring	master + RegionServers on HDFS
Consistency	tunable, AP default	strong per row (CP)
Query	CQL (no joins)	get/put/scan by row key (Phoenix for SQL)

	SQL	NoSQL	NewSQL
Scaling	vertical + replicas	horizontal built in	horizontal built in
Consistency	ACID	often eventual/tunable	ACID (serializable)
Examples	PostgreSQL, MySQL	Redis, MongoDB, Cassandra, Neo4j	Spanner, CockroachDB, TiDB

In the interview

Q: Four NoSQL families? **A:** Key-value (Redis), document (MongoDB), wide-column (Cassandra), graph (Neo4j); each trades joins/constraints for speed on one access pattern and scale-out.

Q: When is Postgres JSONB enough? **A:** When you need flexible attributes plus joins, constraints and transactions at a scale one Postgres (or Citus) handles. Index a JSONB field for ranges with a B-tree expression index on `((doc->>'f')::type)`.

Q: Model "latest 50 messages per chat" in Cassandra. **A:** Partition key `chat_id` (+ time bucket for huge chats), clustering `sent_at DESC`, `LIMIT 50`.

Q: Why are deep joins slow in SQL for graphs? **A:** Each hop is another join through the link table's index and intermediate results multiply; a graph DB follows edge pointers.

Q: Rate limiter? **A:** `INCR key` + `EXPIRE` for a fixed window, or a sorted set of timestamps for a sliding window.

Q: Why is a global write slower in Spanner/CockroachDB? **A:** A majority of replicas, possibly in other regions, must acknowledge; Spanner adds commit-wait.

Q: Design "drivers near me". **A:** Driver locations in a geohash/H3 cell index (Redis GEO or in-memory grid) updated every few seconds; query neighbouring cells, rank by exact distance.

Q: Spark vs MapReduce? **A:** Spark keeps intermediate data in memory instead of writing to HDFS after every step.

Gotchas

- A cache used as the source of truth without persistence → data lost on restart; forgetting TTLs → eviction drops useful keys.
- Huge values or hot keys served by one shard.
- Embedding unbounded arrays (all comments ever) in one document.
- Designing Cassandra tables around entities instead of queries; unbounded partitions.
- Adding a store for fashion rather than a requirement → operational cost and cross-store consistency problems.

Must-know — Elasticsearch/ELK and Redis internals (§16.11)

- **Inverted index** — term → posting list of documents; AND = intersect lists. **Analyzer** = char filters + tokenizer + token filters (lowercase, stop words, stemming), run at index *and* query time. `text` (analysed) vs `keyword` (exact, sort, aggregate).
- **BM25** — default ranking: term frequency (saturating) × inverse document frequency × length normalisation.
- **Shards** — index = N primary shards (fixed; `hash(_id) % N`) + replicas; query-then-fetch scatter-gather; deep paging → `search_after`.
- **Near-real-time** — immutable Lucene segments; **refresh** every 1 s makes new docs searchable; translog for durability; merges in background.
- **ELK** — Elasticsearch + Logstash + Kibana (+ Beats/Elastic Agent); time-based indices with lifecycle management. OpenSearch = the fork.
- **Pairing** — DB is the source of truth; feed the index with CDC (Debezium → Kafka) or an outbox, never dual writes.
- **Redis event loop** — one command thread + epoll/kqueue; every command atomic; one slow command (`KEYS *`, huge `DEL`) blocks all → `SCAN`, `UNLINK`. I/O threads (6.0+) only for sockets.
- **Encodings** — listpack when small → hash table / quicklist / skip list; sorted set = skip list + hash (O(log n) rank).
- **Persistence** — RDB = `fork()` + copy-on-write snapshot (lose minutes); AOF `everysec` (lose ~1 s); often both.
- **Scale** — async replication (+ Sentinel failover); Cluster = 16,384 slots, `CRC16(key) mod 16384`, `MOVED`/`ASK`, hash tags `{user:42}` for multi-key ops.
- **Eviction** — `maxmemory-policy`: `noeviction` (default), `allkeys-lru`/`-lfu` for caches, `volatile-*`; approximated by sampling 5 keys.

Q: Why is Elasticsearch near-real-time? **A:** Documents land in an in-memory buffer and become searchable only when a refresh (1 s default) opens a new segment.

Q: Why is single-threaded Redis fast? **A:** RAM, cheap O(1)/O(log n) commands, no locks or context switches, and I/O multiplexing over thousands of sockets.

Q: RDB vs AOF? **A:** RDB snapshots are compact and fast to load but lose minutes; AOF `everysec` loses about a second but replays slower.

TL;DR

NoSQL families trade relational features for flexible models, scale-out and access-pattern speed; NewSQL keeps SQL + ACID over consensus-replicated shards at a latency cost. Choose by access pattern and consistency; Postgres (JSONB, partitions, BRIN, GiST, FTS, Citus) goes far.

17. Schema Design Case Studies

One design template applied to thirteen systems (hospital, hotel, bank, Instagram, e-commerce, food delivery, library, tickets, warehouse, Quora, TikTok, Airbnb, Snapchat) and the patterns they share.

Must-know

- **The template** — requirements & the questions it must answer → entities, cardinality & participation → ER diagram → tables & constraints → key queries → indexes → design review (normal forms, deliberate redundancy, what it cannot do) → scaling notes.
- **Rules in the schema** — `CHECK` for ranges and status/column consistency (`sold_needs_booking`), partial unique index for "one open X", **exclusion constraint** for "no overlapping X", unique key for "once per user".
- **Type vs instance** — `titles`/`copies`, `room_types`/`rooms`, `seats`/`show_seats`: users choose a kind, you track the item.
- **Current state vs history** — status column + events table, or a **ledger**; "when did…?" needs history rows.
- **Snapshot** values that must not change later (price paid, address shipped to, nightly price) — intentional, not a normalization bug.
- **Soft delete** rows that history still references.
- **Deliberate denormalization** — counters, totals, per-night inventory: update in the same transaction (or trigger) and keep a reconciliation query.
- **Scarce items** — a **guarded** `UPDATE … WHERE status = 'free' RETURNING …` in a short transaction is enough; check the returned row count; `SKIP LOCKED` for "any available".
- **At scale** — relational core for money, bookings and identity; media → object storage + CDN; feeds and counters → caches; events → Kafka/logs; search → search engine (fed by CDC); short-lived data → TTL stores or dropped time partitions.

Key facts & numbers

Situation	Rule
Total participation	FK `NOT NULL`
1:1	FK + `UNIQUE` (or shared PK)
M:N	junction table; pairing attributes (dosage, seat price) live there
Money	`numeric`, never float or int
Ordering events	`timestamptz`, not `date`
Codes with letters/leading zeros	`text`, never `int`
Names `user`, `order`, `group`, `transaction`, `show`	reserved in some dialects → `app_user`, `customer_order`, `screening`
Passwords	slow salted hash, never the password

Pattern	Watch out for
Junction table (PK = two FKs)	index the *second* column for reverse lookups
Status enum + history	enforce allowed values (ENUM/CHECK/lookup)
Soft delete	every query filters; unique needs `WHERE deleted_at IS NULL`
Audit / ledger	cached balance + reconciliation
Polymorphic association	`type + id` loses FKs → one nullable FK per parent + CHECK, or a supertype
Self-reference	recursive CTEs; guard cycles
Overlap prevention	exclusion constraint (PG) or lock-and-check elsewhere
Time partitioning / TTL	include the partition key in WHERE

Case	Key design point
Hospital	appointment = junction with own attributes; prescription_items holds dose; phones multivalued
Hotel	`EXCLUDE USING gist (room_id WITH =, stay WITH &&)` (needs `btree_gist`); `room_inventory` per night with `CHECK (reserved <= total)`
Bank	`ledger_entries` with a shared `transfer_id` so both sides of a transfer are named; entries sum to zero; cached balance recomputed nightly
Instagram	`follows (follower_id, followee_id)` PK + index on `followee_id`; `posts (user_id, created_at DESC, post_id DESC)`; `likes` PK `(post_id, user_id)`
Tickets	seat hold = guarded UPDATE with `hold_until`; sold requires booking (CHECK)
Quora / TikTok / Airbnb / Snapchat	relational core + cache + search + object storage/CDN + event log; Snapchat snaps in TTL stores

Code to remember

```sql
-- hold seats only if still free (or hold expired)
UPDATE show_seats
SET status = 'held', held_by = :user, hold_until = now() + interval '10 minutes'
WHERE show_id = :show AND seat_id = ANY(:seats)
  AND (status = 'free' OR (status = 'held' AND hold_until < now()))
RETURNING seat_id;        -- fewer rows than asked → release and retry

-- no overlapping stays for one room
ALTER TABLE reservation_rooms
  ADD EXCLUDE USING gist (room_id WITH =, stay WITH &&);
```

In the interview

Q: How do you approach "design the schema for X"? **A:** Clarify use cases and scale, pick entities and relationships, sketch the ERD, turn it into tables with keys and constraints, check it against the top queries, index those queries, then say what I'd partition, cache or move to another store as it grows.

Q: How do you know the design is right? **A:** Every core query is simple and index-friendly, every must-never-break rule is a constraint or inside one transaction, and no fact is stored twice without a reason.

Q: Prevent double booking? **A:** UNIQUE only catches identical values, not overlapping ranges: exclusion constraint in PG, or lock and check inside the booking transaction; for seats, a guarded

UPDATE.

Q: Model a bank transfer. **A:** Append-only ledger entries (debit and credit) sharing a transfer id in one transaction; balance as a cached total with a reconciliation job.

Q: When do you denormalize? **A:** For a measured hot read (feed, counter), with the copy updated in the same transaction/trigger and a reconciliation check; beware lock contention on hot counter rows.

Q: Instagram at scale? **A:** State the single-node design first; then shard by user, cache feeds and counters, media in object storage + CDN, search in a search engine.

Gotchas

- One table doing two jobs (type and instance).
- Storing only current state → can't answer "when did…?".
- Recomputing from a mutable price → historical totals change; snapshot instead.
- Polymorphic `target_type + target_id` → no FKs.
- A cached balance with no reconciliation → silent drift.
- Designing for Instagram scale on day one.

TL;DR

Use the same template every time; put rules in the schema (CHECK, partial unique, exclusion); separate type from instance and state from history; snapshot volatile values; denormalize deliberately; guarded UPDATEs for scarce items; relational core + specialised stores at scale.

18. SQL Interview Problems I

A five-step method, then 42 warm-up to medium problems (filtering, aggregation, joins, anti-joins, conditional aggregation, dates, ratios, first windows). Below: the method, the pattern table and the most representative problems.

Must-know

- **Five steps** — (1) **clarify**: grain and keys of each table, what NULLs and duplicates mean, whether zero-activity rows and ties appear, exact output; (2) **sketch** a few sample rows and the expected result; (3) **build** in evaluation order with one CTE per idea; (4) attack **edge cases**; (5) **verify** and discuss performance (indexes on filter/join columns, sargable predicates, pre-aggregation).
- **Filtering** — `WHERE` keeps only TRUE; `IS DISTINCT FROM` when NULLs must count; anchor and escape regexes; count characters (`char_length`), not bytes (MySQL `LENGTH` = bytes).
- **Aggregation** — filter before grouping; `count(col)` skips NULLs (zero counts after LEFT JOIN); an aggregate over nothing returns **one NULL row**, while `LIMIT 1` over nothing returns **no row**.
- **Joins** — group by keys, not names; "every pair" grids = `CROSS JOIN` + `LEFT JOIN`; prefer `EXISTS` for "has at least one"; `WITH TIES`/`RANK` for fair top-1.
- **Anti-joins** — `NOT EXISTS` by default; `LEFT JOIN … IS NULL` with right-table filters in `ON`; `NOT IN` + NULL returns nothing.
- **Dates** — half-open ranges; `date − date` = days (+1 for inclusive); `date − row_number()` = consecutive runs; "as of a date" = latest row `<=` the date + a default.
- **Ratios** — `100.0 *` or cast before dividing, `NULLIF` the denominator, round last; weighted averages when rows carry quantities.

Key facts & numbers

Risk	Symptom	Fix
NULL in comparison	rows vanish; `NOT IN` empty	`IS DISTINCT FROM`, `NOT EXISTS`, `COALESCE`
Entities with no rows	zero-order customers missing	LEFT JOIN from the entity table, `count(col)`
Duplicates	counts too high	`DISTINCT`, `count(DISTINCT)`, check join grain
Fan-out	sums doubled	aggregate before joining
Ties	"the winner" is two people	`RANK` / `DENSE_RANK`, `WITH TIES`
Integer division	`1/3 = 0`	`100.0 *` or `::numeric`
Division by zero	error	`NULLIF(d, 0)`
"NULL if none"	zero rows returned	wrap in aggregate / scalar subquery
Date boundaries	last day or 29 Feb lost	`>= start AND < next_start`

Pattern (problems)	Key SQL feature
Filtering (P1–P8)	`OR` / `AND`, `%`, `IS DISTINCT FROM`, `char_length`, regex `~`, row-level `CASE`
Aggregation (P9–P16)	`HAVING count(*) = 1`, `count(*)` vs `count(col)`, `count(DISTINCT)`, `string_agg(DISTINCT … ORDER BY)`
Joins (P17–P22)	inner, self-join start→end, `CROSS JOIN` grid + LEFT JOIN, `WITH TIES`, `EXISTS`
Anti-joins (P23–P26)	`NOT EXISTS` + half-open range, `LEFT JOIN … IS NULL` (filter in ON), `EXCEPT`, NOT IN trap
Conditional aggregation (P27–P28)	`FILTER`, `SUM(CASE … ELSE 0)`, `UNION ALL` then pivot
Dates (P29–P33)	half-open months, `date - date`, `date - row_number()`, `DISTINCT ON` as-of + default
Ratios (P34–P38)	cast before dividing, `NULLIF` + `COALESCE`, `sum() OVER ()` share, weighted avg, retention
Windows (P39–P42)	`DISTINCT … OFFSET 1`, `DISTINCT ON` / `row_number() = 1`, running `sum() OVER`, `avg() OVER (PARTITION BY)`

Code to remember

Each: problem → key idea → query.

P9 Largest number that appears once (NULL if none) → group, keep singles, `max` over them (aggregate of nothing = NULL row).

```
SELECT max(n) AS num
FROM (SELECT n FROM nums GROUP BY n HAVING count(*) = 1) AS singles;
```

P20 Exam sittings per student × subject, zeros included → build the grid with CROSS JOIN, LEFT JOIN the facts, count a right-side column.

```sql
SELECT st.student_id, su.subject_name, count(e.subject_name) AS attended
FROM student st CROSS JOIN subject su
LEFT JOIN exam_sitting e
  ON e.student_id = st.student_id AND e.subject_name = su.subject_name
GROUP BY st.student_id, su.subject_name
ORDER BY 1, 2;
```

P26 Reps who never sold to company 'RED' → correlated `NOT EXISTS` (NOT IN breaks once an order has a NULL rep).

```sql
SELECT r.name FROM sales_rep r
WHERE NOT EXISTS (SELECT 1 FROM orders o JOIN company c USING (com_id)
                  WHERE o.rep_id = r.rep_id AND c.name = 'RED');
```

P32 Runs of 3+ consecutive absent days → `date − row_number()` is constant inside a run.

```sql
SELECT student_id, min(att_date) AS run_start
FROM (SELECT student_id, att_date,
             att_date - (row_number() OVER (PARTITION BY student_id ORDER BY att_date))::int AS grp
      FROM attendance WHERE status = 'Absent') t
GROUP BY student_id, grp HAVING count(*) >= 3;
```

P33 Price on 15 March 2024 (10 before any change) → latest row on or before the date + default.

```sql
WITH latest AS (SELECT DISTINCT ON (product_id) product_id, new_price
                FROM price_change WHERE effective_on <= DATE '2024-03-15'
                ORDER BY product_id, effective_on DESC)
SELECT p.product_id, COALESCE(l.new_price, 10) AS price
FROM (SELECT DISTINCT product_id FROM price_change) p LEFT JOIN latest l USING (product_id);
```

P35 Ad CTR = clicks / (clicks + views), 0 if neither → `FILTER` counts, `NULLIF` the denominator, `COALESCE` to 0.

```sql
SELECT ad_id,
  round(COALESCE(100.0 * count(*) FILTER (WHERE action = 'Clicked')
    / NULLIF(count(*) FILTER (WHERE action IN ('Clicked','Viewed')), 0), 0), 2) AS ctr
FROM ad_action GROUP BY ad_id ORDER BY ctr DESC, ad_id;
```

P38 Share of players back the day after first login → first day per player, LEFT JOIN on `first + 1`, count matches / all.

```sql
WITH f AS (SELECT player_id, min(played_on) AS first_on FROM activity GROUP BY player_id)
SELECT round(count(a.player_id)::numeric / count(*), 2) AS fraction
FROM f LEFT JOIN activity a
  ON a.player_id = f.player_id AND a.played_on = f.first_on + 1;
```

P39 Second-highest distinct salary, NULL if none → scalar subquery so "no row" becomes NULL.

```sql
SELECT (SELECT DISTINCT salary FROM emp
        ORDER BY salary DESC OFFSET 1 LIMIT 1) AS second_highest_salary;
```

P41 Games played so far per player → running window sum.

```sql
SELECT player_id, played_on,
       sum(games) OVER (PARTITION BY player_id ORDER BY played_on) AS games_so_far
FROM activity;
```

In the interview

Q: How do you approach a SQL problem? **A:** Pin down grain, keys, NULLs, duplicates, zero rows and ties; sketch sample rows and expected output; build in evaluation order with CTEs; run the edge-case checklist; verify and discuss indexes.

Q: Your query returns nothing for a new customer — right? **A:** Depends on the spec; if they should appear with 0, start from the customer table with LEFT JOIN and count/COALESCE a right-side column.

Q: Would this be fast on 100M rows? **A:** Index the filter/join columns, keep predicates sargable (no functions on indexed columns), pre-aggregate before joining.

Q: Election winner? **A:** `ORDER BY count(*) DESC FETCH FIRST 1 ROWS WITH TIES` (or `rank() = 1`) — say how ties are handled.

Q: Loan share of grand total? **A:** `amount / sum(amount) OVER ()` or a scalar subquery for the total.

Gotchas

- Typing before knowing each table's grain.
- Selecting a non-grouped column (PG rejects; MySQL with `ONLY_FULL_GROUP_BY` off returns an arbitrary value).
- Right-table filter of a LEFT JOIN in WHERE → inner join.
- Omitting `ORDER BY` when the question asks for an order.
- MySQL `GROUP_CONCAT` output capped by `group_concat_max_len` (1024 bytes); SQL Server `STRING_AGG` has no DISTINCT.

TL;DR

Clarify, sketch, build in CTE layers, attack edge cases (NULL, zeros, duplicates, fan-out, ties, integer division, dates), verify. Most warm-ups are WHERE + GROUP BY/HAVING + LEFT JOIN/NOT EXISTS + FILTER + one window.

19. SQL Interview Problems II

41 medium and hard problems: Nth highest and top-N, running totals, gaps and islands, static and dynamic pivots, relational division, retention and rates. Below: the three habits, the pattern catalogue and the most representative problems.

Must-know

- **Build in layers** — one CTE per idea (one row per user-day → rank within day → keep rank 1); run each layer before adding the next.
- **Say the tie rule out loud** — it picks `ROW_NUMBER` (exactly one winner), `RANK` (keep ties) or `DENSE_RANK` (top N distinct values).
- **Hunt the empty case** — players with no wins, days with no visits: 0, NULL, or absent?
- **Islands** — `value − row_number()` or the **difference of two row numbers** (`row_number() OVER (PARTITION BY g ORDER BY ts) − row_number() OVER (PARTITION BY g, val ORDER BY ts)`) is constant within a run; de-duplicate first.
- **Pairs stored once** (friendships, calls) → unfold with `UNION ALL` so each side gets a row.
- **Relational division** ("took all", "bought every") → `HAVING count(DISTINCT y) = (SELECT count(*) FROM all_y)` or double `NOT EXISTS`.

- **Dynamic pivot** — generate SQL with `format()` (`%I` identifiers, `%L` literals) and `EXECUTE` it into a view, or `\gexec`. Unpivot with `LATERAL (VALUES …)` or `jsonb_each_text(to_jsonb(t) - 'id')`.
- **PG traps** — integer division truncates, division by zero errors, `NOT IN` + NULL is empty, `count(*)` after LEFT JOIN counts the padding row, `DESC` sorts NULLs first.

Key facts & numbers

Pattern	Recognise it by	Template (PostgreSQL)
Nth highest	"second highest", "NULL if none"	`SELECT (SELECT DISTINCT v FROM t ORDER BY v DESC OFFSET n-1 LIMIT 1)`
Top-N per group	"top 3 in each"	`dense_rank() OVER (PARTITION BY g ORDER BY v DESC) <= n` in a CTE
First/last per group	"first session", "latest status"	`DISTINCT ON (g) … ORDER BY g, ts`
All tied for max	"most … include ties"	`WHERE cnt = (SELECT max(cnt) FROM c)` or `rank() = 1`
Capacity cut-off	"waitlist", "fits in the lift"	`row_number()` or `sum(w) OVER (ORDER BY pos)` vs limit
Running total	"so far", "to date"	`sum(v) OVER (PARTITION BY g ORDER BY ts)`
Vs previous row	"improved", "change since"	`v - lag(v) OVER (PARTITION BY g ORDER BY ts)`
Consecutive days	"N days in a row"	`GROUP BY g, day - row_number()…::int HAVING count(*) >= n`
Runs of equal values	"longest streak"	difference of two row numbers
Static pivot	fixed output columns	`max(v) FILTER (WHERE k = 'a') AS a … GROUP BY id`
Pivot of lists	names under each region	`row_number() OVER (PARTITION BY k ORDER BY v)` then `GROUP BY rn` + FILTER
Unpivot	wide → rows	`CROSS JOIN LATERAL (VALUES ('a', a), ('b', b))`
Pair stored once	friends, calls	`SELECT a, b FROM t UNION ALL SELECT b, a FROM t`
Relational division	"every"	`HAVING count(DISTINCT y) = (SELECT count(*) …)`
Vs group average	"above dept average"	`v > avg(v) OVER (PARTITION BY g)`
Uniqueness	"shared", "unique location"	`count(*) OVER (PARTITION BY k) > 1`
Retention	"came back next day"	`min(day)` per user, then `avg((EXISTS (… day = first + 1))::int)`
Histogram with empty buckets	"0 to max"	`generate_series(0, max) n LEFT JOIN counts ON c = n`
Rate	"percentage", "share"	`round(100.0 * count(*) FILTER (WHERE c) / nullif(count(*), 0), 2)`
String from rows	"build the equation"	`string_agg(expr, sep ORDER BY k)`

Code to remember

Top 3 distinct scores per subject (ties included) → `dense_rank` in a CTE, filter outside.

```sql
WITH r AS (SELECT subject_id, pupil, score,
                  dense_rank() OVER (PARTITION BY subject_id ORDER BY score DESC) AS dr
           FROM results)
SELECT * FROM r WHERE dr <= 3 ORDER BY subject_id, score DESC, pupil;
```

Confirmed or waitlisted (seats per flight, by booking time) → `row_number()` compared with capacity.

```sql
SELECT b.passenger_id,
       CASE WHEN row_number() OVER (PARTITION BY b.flight_id ORDER BY b.booked_at) <= f.seats
            THEN 'Confirmed' ELSE 'Waitlist' END AS status
FROM bookings b JOIN flights f USING (flight_id);
```

Last person who fits in a 500 kg lift → running weight in queue order, keep the last under the limit.

```sql
WITH b AS (SELECT name, turn, sum(kg) OVER (ORDER BY turn) AS load_kg FROM lift_queue)
SELECT name FROM b WHERE load_kg <= 500 ORDER BY turn DESC LIMIT 1;
```

Longest winning streak per player (0 if never won) → difference of two row numbers labels each run.

```sql
WITH t AS (SELECT player_id, outcome,
             row_number() OVER (PARTITION BY player_id ORDER BY played_on)
           - row_number() OVER (PARTITION BY player_id, outcome ORDER BY played_on) AS island
           FROM games),
w AS (SELECT player_id, count(*) AS len FROM t WHERE outcome = 'win' GROUP BY player_id, island)
SELECT p.player_id, coalesce(max(w.len), 0) AS longest
FROM (SELECT DISTINCT player_id FROM games) p LEFT JOIN w USING (player_id) GROUP BY p.player_id;
```

Names by region side by side → number rows per region, group by that number, one FILTER per column.

```sql
WITH n AS (SELECT name, region, row_number() OVER (PARTITION BY region ORDER BY name) AS rn
           FROM attendees)
SELECT max(name) FILTER (WHERE region = 'America') AS america,
       max(name) FILTER (WHERE region = 'Asia')    AS asia,
       max(name) FILTER (WHERE region = 'Europe')  AS europe
FROM n GROUP BY rn ORDER BY rn;
```

Students enrolled in every course (relational division) → compare distinct count with the catalogue size.

```sql
SELECT student_id FROM enrolments
GROUP BY student_id
HAVING count(DISTINCT course_id) = (SELECT count(*) FROM catalogue);
```

Same person first and last call of the day → unfold pairs with UNION ALL, number from both ends.

```sql
WITH legs AS (SELECT caller AS person, callee AS other, called_at FROM phone_calls
              UNION ALL SELECT callee, caller, called_at FROM phone_calls),
e AS (SELECT person, called_at::date AS day, other,
        row_number() OVER (PARTITION BY person, called_at::date ORDER BY called_at)      AS s,
        row_number() OVER (PARTITION BY person, called_at::date ORDER BY called_at DESC) AS f
      FROM legs)
SELECT DISTINCT person FROM e GROUP BY person, day
HAVING max(other) FILTER (WHERE s = 1) = max(other) FILTER (WHERE f = 1);
```

Day-1 retention per install day → first day per player, LEFT JOIN the next day, ratio.

```
SELECT f.install_day, count(*) AS installs,
       round(count(p.player_id)::numeric / count(*), 2) AS day1_retention
FROM (SELECT player_id, min(play_day) AS install_day FROM plays GROUP BY player_id) f
LEFT JOIN plays p ON p.player_id = f.player_id AND p.play_day = f.install_day + 1
GROUP BY f.install_day;
```

Transactions-per-visit histogram incl. empty buckets → count per visit, then `generate_series` of buckets LEFT JOIN.

```
WITH pv AS (SELECT v.customer_id, v.visit_day, count(t.txn_day) AS txns
            FROM branch_visits v LEFT JOIN branch_txns t
              ON t.customer_id = v.customer_id AND t.txn_day = v.visit_day
            GROUP BY v.customer_id, v.visit_day)
SELECT n AS txns, count(pv.txns) AS visits
FROM generate_series(0, (SELECT max(txns) FROM pv)) n LEFT JOIN pv ON pv.txns = n
GROUP BY n ORDER BY n;
```

Ride cancellation rate per day, unbanned riders and drivers only → join users twice (as rider and as driver), average a boolean.

```
SELECT r.requested_on AS day, round(avg((r.status <> 'completed')::int), 2) AS rate
FROM rides r
JOIN app_users ru ON ru.user_id = r.rider_id  AND NOT ru.banned
JOIN app_users du ON du.user_id = r.driver_id AND NOT du.banned
WHERE r.requested_on BETWEEN '2026-10-01' AND '2026-10-03'
GROUP BY r.requested_on;
```

In the interview

Q: ROW_NUMBER, RANK or DENSE_RANK for top-N? **A:** Exactly N rows → ROW_NUMBER with a tie-breaker; N places with ties → RANK; N distinct values → DENSE_RANK. State the tie rule first.

Q: Find the longest streak. **A:** Label runs with the difference of two row numbers (or a running count of non-wins), group by the label, take the max; LEFT JOIN from all players for 0.

Q: How do you pivot when columns are unknown? **A:** Generate the SQL with `format(%I, %L)` + `string_agg` and EXECUTE it (into a view), or return JSON; SQL needs the column list at parse time.

Q: "Customers who bought every product"? **A:** Relational division: GROUP BY customer HAVING count(DISTINCT product) = total products, or double NOT EXISTS.

Q: Friend-of-friend suggestions / mutual pairs? **A:** Unfold the pair table with UNION ALL, self-join, exclude existing friends with NOT EXISTS.

Gotchas

- Forgetting to de-duplicate before an islands query (two logins on one day break the arithmetic).
- `RANGE` default frame on a tied order key → running totals jump.
- Denominator choice for rates (which rows count) decided silently → say it.
- Players/days with no rows vanish → start from the entity or `generate_series` and LEFT JOIN.

TL;DR

Layer CTEs, state tie rules, hunt empty cases. Ranking → top-N/first-per-group; windows → running totals and lag; row-number differences → islands and streaks; FILTER → pivots; UNION ALL → pairs; HAVING count = total → division; generate_series → empty buckets.

20. Cheat Sheets, Interview Q&A and Glossary

The book on a few pages: evaluation order, one sheet per area, decision tables, the five-database dialect table, the pattern catalogue (already condensed in §19 above), 54 interview Q&As (the best of them are in the Top 25 at the front) and a glossary of 298 terms.

Must-know

- **Evaluation order explains most errors** — FROM → WHERE → GROUP BY → HAVING → SELECT (windows) → DISTINCT → set operations → ORDER BY → LIMIT.
- **NULL** — `WHERE` keeps only TRUE; `NOT IN` with a NULL returns nothing; aggregates skip NULLs; `sum` over no rows is NULL.
- **PostgreSQL arithmetic** — `7 / 2 = 3`; write `100.0 *` before dividing and guard with `NULLIF(x, 0)`; `1/0` is an error (22012).
- **Joins** — right-table filters of a LEFT JOIN go in `ON`; `NOT EXISTS` is the safe anti-join; watch fan-out.
- **Windows** — `row_number` / `rank` / `dense_rank` differ only on ties; default frame with ORDER BY is `RANGE … CURRENT ROW`.
- **Design** — entities, keys, relationships first; normalize to 3NF/BCNF; denormalize only with a reason and a maintenance path.
- **Indexes** — B-tree by default, equality before range columns, no functions on indexed columns, keyset over deep OFFSET, compare estimated vs actual rows.
- **Transactions** — PG default Read Committed; Repeatable Read = snapshot isolation (write skew possible); Serializable (SSI) needs retries on 40001.
- **Dialects** differ most on paging, concatenation, NULL sorting, upsert, `RETURNING`, integer division and transactional DDL.
- **Verified** — about 100 PostgreSQL claims and 28 SQLite claims on the sheets are asserted by runnable scripts.

Key facts & numbers

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
First n rows	`LIMIT n` / `FETCH FIRST`	`LIMIT n`	`TOP (n)` / `OFFSET…FETCH`	`LIMIT n`	`FETCH FIRST` (12c), `ROWNUM`
Unquoted names fold to	lower	OS-dependent (tables)	kept	kept	UPPER
Concat with NULL	`||` → NULL; `concat()` skips	`CONCAT` → NULL; `||` = OR	`+`; `CONCAT` treats NULL as ''	`||`	`||` treats NULL as ''
`''` is NULL?	no	no	no	no	**yes**
`7 / 2`	3	3.5000 (`DIV` = 3)	3	3	3.5
`1 / 0` in SELECT	error	NULL + warning	error	NULL	error
NULL-safe equal	`IS NOT DISTINCT FROM`	`<=>`	`IS NOT DISTINCT FROM` (2022)	`IS`	`DECODE`
NULLs in ASC sort	last	first	first	first	last
`GREATEST(1, NULL)`	1	NULL	1 (2022)	`max(a,b)` → NULL	NULL
UNIQUE with many NULLs	allowed	allowed	**one**	allowed	allowed
LIKE case-sensitive	yes	no (default)	no (default)	no (ASCII)	yes
FULL JOIN	yes	no	yes	3.39	yes
LATERAL	yes	8.0.14	`APPLY`	no	yes
String agg	`string_agg`	`GROUP_CONCAT`	`STRING_AGG` (2017)	`group_concat`	`LISTAGG`
`FILTER (WHERE)`	yes	no	no	3.30	no
`avg(int)`	numeric	decimal	**int**	float	NUMBER
Median	`percentile_cont`	–	window only	–	`MEDIAN`
ROLLUP/CUBE/SETS	all	`WITH ROLLUP`	all	–	all
Recursive CTE	`WITH RECURSIVE`	`WITH RECURSIVE`	`WITH`	`WITH RECURSIVE`	`WITH`
`DISTINCT ON`	yes	–	–	–	–
`QUALIFY`	–	–	–	–	26ai
Upsert	`ON CONFLICT`, `MERGE` (15)	`ON DUPLICATE KEY UPDATE`	`MERGE`	`ON CONFLICT` (3.24)	`MERGE`
Return changed rows	`RETURNING`	–	`OUTPUT`	`RETURNING` (3.35)	`RETURNING INTO`

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Transactional DDL	yes	no	mostly	yes	no
Default isolation	Read Committed	Repeatable Read	Read Committed	Serializable	Read Committed
"Serializable" means	SSI	2PL (locking reads)	2PL (or SNAPSHOT)	one writer	snapshot isolation
Materialized views	manual REFRESH	—	indexed views	—	incremental refresh
Table storage	heap	clustered on PK	heap or clustered	rowid B-tree	heap (IOT optional)
Show plan	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE`	actual plan	`EXPLAIN QUERY PLAN`	`DBMS_XPLAN`
Add 3 days	`d + 3`	`DATE_ADD(d, INTERVAL 3 DAY)`	`DATEADD(day,3,d)`	`date(d,'+3 days')`	`d + 3`

Decision	Pick
Has a match / no match	`EXISTS` / `NOT EXISTS`
One row per group	`row_number() = 1` or `DISTINCT ON`
Index type	B-tree; GIN (JSONB, arrays, FTS, trigram); GiST (ranges, geometry, exclusion); BRIN (append-only time)
Isolation	Read Committed; RR READ ONLY for reports; Serializable + retry for cross-row invariants
Scaling	index → bigger box → read replicas/cache → partition → shard

In the interview

Q: What are DDL, DML, DQL, DCL, TCL? **A:** Structure (`CREATE/ALTER/DROP`), data changes (`INSERT/UPDATE/DELETE`), queries (`SELECT`), rights (`GRANT/REVOKE`), transactions (`BEGIN/COMMIT/ROLLBACK`).

Q: CTE vs subquery vs view vs materialized view? **A:** Named step in one statement / inline / stored query always current / stored result that must be refreshed.

Q: Backup vs replica? **A:** A replica copies every change, including mistakes, within seconds; a backup is a point-in-time copy you can restore; PITR = base backup + archived WAL.

Q: How do you read EXPLAIN ANALYZE? **A:** Innermost node first; compare estimated vs actual rows, check loops, biggest actual times, sorts spilling to disk, seq scans on big tables with selective filters.

Gotchas

- Porting PG habits: MySQL `7/2 = 3.5`, `||` is OR, DDL commits implicitly; SQL Server `AVG(int)` truncates and UNIQUE allows one NULL; Oracle treats `''` as NULL.
- SQL Server has no `NATURAL`/`USING`, no row-value `IN`; MySQL has no FULL JOIN and no `FILTER`.

TL;DR

Evaluation order, NULL logic, integer division, LEFT JOIN filters in ON, NOT EXISTS, tie-aware ranking, window frames, 3NF, equality-then-range indexes, keyset paging, isolation levels with

retries — plus the dialect differences above.

Last-minute checklist

- [ ] Evaluation order: FROM → WHERE → GROUP BY → HAVING → SELECT (windows) → DISTINCT → set ops → ORDER BY → LIMIT.
- [ ] NULL = unknown: `IS NULL`, `IS DISTINCT FROM`, `COALESCE`; `NOT IN` + NULL = empty → `NOT EXISTS`.
- [ ] `count(*)` counts rows; every other aggregate skips NULLs; `sum` of nothing is NULL.
- [ ] PG `7/2 = 3` → `100.0 *` first; `NULLIF(d, 0)` on every denominator; round last.
- [ ] Half-open date ranges `>= start AND < next`; no functions on indexed columns.
- [ ] LEFT JOIN: right-table filters in ON; count a right-side key, not `*`; aggregate children before joining (fan-out).
- [ ] UNION ALL by default; INTERSECT/EXCEPT for NULL-safe diffs.
- [ ] Windows: rank in a CTE, filter outside; say the tie rule (row_number / rank / dense_rank).
- [ ] Default frame with ORDER BY = RANGE to CURRENT ROW (ties); `last_value` needs a full frame.
- [ ] Islands: `value − row_number()`; streaks: difference of two row numbers; de-duplicate first.
- [ ] Pivot = `max/sum(...) FILTER (WHERE …)` per column; unpivot = `LATERAL (VALUES …)`; division = `HAVING count(DISTINCT) = total`.
- [ ] Recursive CTE: anchor + recursive term, stop condition, cast in anchor, path array for cycles.
- [ ] Keys: super ⊇ candidate ⊇ primary; alternate → UNIQUE; index FK columns in PG.
- [ ] 1:N → FK on many side; M:N → junction; 1:1 → FK + UNIQUE; weak entity → owner key + partial key.
- [ ] 3NF: every X → A has X a superkey or A prime; BCNF: every determinant a superkey; lossless = shared columns are a key.
- [ ] Upsert: `INSERT … ON CONFLICT (k) DO UPDATE SET c = EXCLUDED.c`.
- [ ] B+ tree 3–4 levels; composite = equality first, range last; INCLUDE, partial, expression indexes; GIN/GiST/BRIN.
- [ ] `EXPLAIN (ANALYZE, BUFFERS)`: estimated vs actual rows; pages touched; keyset over OFFSET.
- [ ] Anomalies: dirty read, non-repeatable read, phantom, lost update, write skew; PG RR = snapshot isolation; Serializable = SSI.
- [ ] Retry 40001 and 40P01; lock rows in a consistent order; `SKIP LOCKED` for queues; version column for think time.
- [ ] MVCC: readers don't block writers; VACUUM; no long idle-in-transaction sessions.
- [ ] Parameterize every value; least privilege; RLS `USING` = read, `WITH CHECK` = write; owners bypass RLS.
- [ ] Scale: index → bigger box → replicas + cache → partition → shard (high-cardinality key, consistent hashing with vnodes).
- [ ] Sync vs async replication; R + W > N; CAP = CP or AP under partition; Raft 2f+1; 2PC blocks.
- [ ] Replicas are not backups: base backup + WAL = PITR; test restores.